



Universidad Autónoma de Chihuahua

Facultad de Ingeniería

Detección de la Abeja Reina en una Colmena por Medio del Aprendizaje Profundo

Análisis de Frecuencias Emitidas por el Panal

Estudiante: Leonardo Rafael León Mora.

Director de Tesis: Dr. Daniel Espinobarro Velázquez.

Revisores:

- M.C. Carlos Hugo Larrinúa Pacheco.
- M.I. Joseph Isaac Ramírez Hernández.

Chihuahua, Chih., 21 de abril de 2026.

Índice general

RESUMEN	4
1. INTRODUCCIÓN	5
1.1. Planteamiento del Problema	5
1.2. Justificación	6
1.3. Objetivos	6
1.3.1. Objetivo General	6
1.3.2. Objetivos Específicos	6
1.4. Hipótesis de Trabajo	7
2. MARCO TEÓRICO	8
2.1. Las Abejas Melíferas y su Importancia Ecológica y Económica	8
2.1.1. Rol de las Abejas en la Agricultura y Ecosistemas	8
2.1.2. Declive de Poblaciones y Factores de Riesgo	8
2.1.3. Estructura Social de la Colmena y Rol de la Reina	8
2.1.4. Necesidad de Sistemas de Monitoreo No Invasivos	9
2.2. Monitoreo Acústico de Colmenas: Estado del Arte	9
2.2.1. Fundamentos del Análisis Acústico en Apicultura	9
2.2.2. Comunicación Acústica en Abejas Melíferas	9
2.2.3. Detección de Eventos mediante Análisis Acústico	10
2.2.4. Otros Enfoques de Machine Learning	11
2.2.5. Limitaciones de Estudios Previos y Brechas de Conocimiento	12
2.3. Fundamentos de Procesamiento Digital de Señales de Audio	12
2.3.1. Representación Digital de Señales de Audio	12
2.3.2. Características de Señales de Audio de Colmenas	12
2.3.3. Transformada de Fourier de Tiempo Corto (STFT)	13
2.3.4. Transformada Rápida de Fourier (FFT)	14
2.4. Coeficientes Cepstrales en Frecuencia Mel (MFCC)	14
2.4.1. Motivación y Fundamento Perceptual	14
2.4.2. La Escala Mel	15
2.4.3. Proceso de Extracción de MFCC	15
2.4.4. Configuraciones de MFCC en Estudios de Colmenas	19

2.4.5.	Implementación Práctica	19
2.5.	Aprendizaje Profundo para Clasificación de Audio	20
2.5.1.	Conceptos Fundamentales de Redes Neuronales	20
2.5.2.	Perceptrón Multicapa (MLP)	21
2.5.3.	Regresión Logística	21
2.5.4.	Redes Neuronales Recurrentes (RNN) y LSTM	21
2.5.5.	Redes Neuronales Convolucionales (CNN)	23
2.5.6.	Comparación de Arquitecturas	25
2.6.	Síntesis y Justificación Metodológica	25
2.6.1.	Contexto y Motivación	25
2.6.2.	Estado del Arte en Monitoreo Acústico	25
2.6.3.	Fundamentos Técnicos	25
2.6.4.	Justificación de la Metodología Propuesta	26
3.	METODOLOGÍA	27
3.1.	Procedimiento	27
3.2.	Modelos Involucrados	30
3.2.1.	CNN_1: Arquitectura Base con Activación ReLU	31
3.2.2.	CNN_2: Arquitectura con Activación Sigmoide	33
3.3.	Distribución del Conjunto de Datos	35
3.4.	Instrumentos	35
3.5.	Base de Datos	36
4.	RESULTADOS	37
4.1.	Resultados del Entrenamiento	37
4.2.	Evaluación en el Conjunto de Prueba	37
4.3.	Análisis Temporal de Predicciones	38
5.	DISCUSIÓN	39
5.1.	Interpretación de los Resultados	39
5.1.1.	Comportamiento de CNN_1	39
5.1.2.	Comportamiento de CNN_2	39
5.2.	Análisis Comparativo	39
5.3.	Comparación con el Estado del Arte	39
5.3.1.	Comparación con el Estado del Arte	40
5.4.	Limitaciones y Desafíos Identificados	40
5.4.1.	Generalización	40

5.4.2. Desbalance de Clases	40
5.4.3. Robustez al Ruido	40
5.4.4. Implicaciones Prácticas	40
5.4.5. Direcciones Futuras	41
6. CONCLUSIONES	42
BIBLIOGRAFÍA	43
APÉNDICE A	45
Programa # 1	45
Programa # 2	51
Programa # 3	56
Programa # 3	60

RESUMEN

1. INTRODUCCIÓN

Las abejas melíferas han sido ampliamente estudiadas por su destacada contribución al medio ambiente, proporcionando recursos que son vitales para animales, plantas y seres humanos. Su papel en la polinización es crucial para la biodiversidad y la producción agrícola[1]. Además, la miel, el polen, la jalea real y la cera de abejas son productos altamente valorados por sus propiedades nutricionales y medicinales[2].

Durante las temporadas de mayor producción de miel, como en primavera y verano, las abejas melíferas pueden recorrer distancias de hasta 6 km[3], viajando aproximadamente 3 km de ida y vuelta entre la colmena y las fuentes de alimento, como aquellas flores de donde recolectan polen, néctar o agua. Pero, ¿por qué surge este comportamiento? ¿Cuál es la motivación de las abejas para regresar a la colmena y abastecerla?

La comunicación dentro de la colmena es esencial para la supervivencia de la colonia. Las abejas utilizan una variedad de señales, incluyendo feromonas, zumbidos y otros sonidos, para coordinar sus actividades y mantener el orden social[2]. La presencia de una abeja reina es fundamental para la cohesión del grupo, ya que su ausencia puede conducir a un comportamiento caótico y al eventual colapso de la colonia[4]. Otra forma de comunicación es el *waggle dance*, mediante el cual las abejas pueden informar sobre la ubicación de fuentes de alimento, tomando como referencia los puntos cardinales y trazando un vector en la dirección requerida[5], lo que permite notificar a las demás abejas recolectoras, dando a conocer la dirección y distancia a las fuentes de alimento.

Según Ruvinga[4], “*La colonia está formada por la reina, miles de abejas obreras estériles, unos pocos cientos de zánganos, huevos, larvas y pupas*”. Las abejas trabajan a diario, reali-

zando tareas que se dividen no solo entre machos y hembras, sino también entre abejas jóvenes y adultas[6]. Un panal puede albergar entre 30,000 y 50,000 abejas, aunque este número puede reducirse hasta 5,000 durante aquellas temporadas de menor producción (usualmente verano o invierno). Este comportamiento de abastecimiento constante de la población está estrechamente vinculado a la estructura social de la colmena y, en particular, al papel de la abeja reina, que es la única capaz de procrear[4].

1.1. Planteamiento del Problema

La reina puede poner en promedio hasta 2,000 huevos al día. Las obreras se encargan de construir, defender y abastecer la colmena[2]. Los zánganos cumplen funciones reproductivas y también se cree que ayudan a regular la temperatura del panal, ventilando la colmena con sus alas para crear corrientes de aire dentro de la colmena. Las abejas más jóvenes se limitan a tareas dentro del panal, mientras que las más adultas y en su mayoría hembras, se encargan de actividades externas[6].

Las colmenas están expuestas a múltiples amenazas: insectos invasores, pesticidas, condiciones climáticas e inclusive seres humanos. En sistemas apícolas tradicionales donde se utilizan cajas de madera, cada inspección física implica abrir la colmena y romper el sello de propóleo que las abejas han construido para mantener un ambiente limpio y controlado. Esta intervención frecuente no solo representa una pérdida energética para la colonia, que debe reconstruir el sello de propóleo y reabastecer el panal con respecto a las abejas que mueren en este proceso; sino que además incrementa el riesgo de aplastar accidentalmente a la reina durante la manipulación interna de la colmena, lo que pue-

de conducir al colapso de toda la colonia si no se toma acción inmediata.

Una abeja reina puede ser reemplazada por la colonia mediante la crianza de una nueva reina a partir de larvas seleccionadas que se incuban en jalea real. Siempre y cuando existan suficientes recursos y tiempo para que la nueva reina se desarrolle completamente, el proceso puede tardar entre 16 y 24 días, dependiendo de las condiciones ambientales y la disponibilidad de alimento[2]. Durante este período, la colonia puede volverse vulnerable a enfermedades, depredadores e inclusive la misma reina, son factores estresantes que pueden afectar la supervivencia de una nueva reina. Como una alternativa, los apicultores pueden introducir una reina comprada o criada en otro lugar en caso de que sea necesario restablecer la estabilidad de la colonia. Pero, la aceptación de una nueva reina por parte de las obreras no está garantizada y puede variar según el comportamiento de la colonia y las condiciones ambientales[4].

1.2. Justificación

Contar con herramientas que permitan identificar la presencia o ausencia de la abeja reina de manera no invasiva resulta de gran utilidad para garantizar la continuidad y productividad de la colmena. Esta investigación propone un enfoque basado en el análisis acústico del pannel, aprovechando técnicas de aprendizaje automático para detectar el estado de la reina. Dado que estudios recientes han demostrado que el sonido dentro de la colmena varía según la presencia o ausencia de la reina[4, 7, 8, 9], se busca validar un sistema que, a partir de grabaciones de audio, identifique estas variaciones de forma automática y eficiente.

1.3. Objetivos

1.3.1. Objetivo General

Crear una red neuronal de análisis acústico basado en aprendizaje automático para detectar:

- Ausencia de la reina.
- Presencia de la reina.
- Aceptación de una nueva reina.
- Rechazo de una nueva reina.

Esto con el fin de proporcionar a los apicultores una herramienta no invasiva que permita monitorear la salud y estabilidad de sus colonias de manera eficiente y oportuna.

Esto con el objetivo de que en un futuro se pueda desarrollar un dispositivo portátil y de bajo costo que pueda ser utilizado por apicultores para monitorear sus colmenas en tiempo real, utilizando técnicas de aprendizaje automático para analizar las grabaciones de audio y detectar el estado de la reina. Este dispositivo podría alertar a los apicultores sobre cambios en el estado de la reina, permitiéndoles tomar medidas preventivas para garantizar la salud y productividad de sus colonias.

1.3.2. Objetivos Específicos

- 🔊 Realizar una revisión bibliográfica sobre la estructura de las colonias de abejas melíferas y los efectos de la ausencia de la reina.
- 🔊 Procesar y organizar un conjunto de datos de audio de colmenas etiquetado con el estado de la reina.
- 🔊 Extraer características acústicas relevantes, como los coeficientes cepstrales en la escala Mel (MFCC), a partir de las grabaciones.
- 🔊 Entrenar y evaluar modelos de aprendizaje automático para clasificar el estado de la reina basado en características extraídas.
- 🔊 Validar la efectividad del sistema en términos de precisión, sensibilidad y especificidad.

1.4. Hipótesis de Trabajo

Dado que el comportamiento sonoro de una colmena varía según el estado de su reina, se espera que un sistema basado en el análisis de características acústicas sea capaz de detectar, con un grado aceptable de precisión si:

1. la reina está presente en la colonia, lo que se reflejará en un patrón acústico característico.
2. la reina está ausente, lo que se manifestará en una disminución de ciertas frecuencias y un aumento de otras.
3. la colonia ha rechazado una nueva reina, lo que se evidenciará en un patrón acústico discordante y caótico.
4. la colonia ha aceptado una nueva reina, lo que se traducirá en un patrón acústico armonioso similar al de una colonia con reina presente.

2. MARCO TEÓRICO

2.1. Las Abejas Melíferas y su Importancia Ecológica y Económica

2.1.1. Rol de las Abejas en la Agricultura y Ecosistemas

Las abejas melíferas (*Apis mellifera*) constituyen uno de los grupos de polinizadores más importantes a nivel mundial, visitando más del 90 % de los 107 principales tipos de cultivos globales [10]. Su labor de polinización contribuye directamente a la producción de aproximadamente el 75 % de los cultivos de importancia alimentaria, incluyendo la mayoría de frutas, semillas y nueces, así como cultivos de alto valor comercial como café, cacao y canola [11].

El valor económico de los servicios de polinización animal se estima entre USD \$235-577 mil millones anuales a nivel global [11], representando aproximadamente el 9.5 % del valor total de la producción agrícola destinada al consumo humano. Sin embargo, más allá del valor monetario directo, la polinización por abejas es fundamental para la seguridad alimentaria global: se estima que entre 5-8 % de la producción mundial de cultivos se perdería sin servicios de polinización animal [10], lo que requeriría cambios significativos en las dietas humanas y una expansión desproporcionada de tierras agrícolas.

2.1.2. Declive de Poblaciones y Factores de Riesgo

En las últimas décadas se ha documentado un declive alarmante en las poblaciones de abejas melíferas en múltiples regiones del mundo, particularmente en Europa occidental y Norteamérica [11]. Este declive ha sido atribuido a

diversos factores interrelacionados, incluyendo:

- **Cambios en el uso del suelo:** La conversión de hábitats semi-naturales a agricultura intensiva reduce la disponibilidad de recursos florales y sitios de anidación [11].
- **Pesticidas:** Particularmente los neonicotinoides, que han demostrado efectos letales y subletales en polinizadores, afectando su navegación, aprendizaje y reproducción [11].
- **Patógenos y parásitos:** El ácaro *Varroa* y virus asociados representan amenazas significativas para la salud de las colonias [11].
- **Cambio climático:** Alteraciones en la sincronización temporal y espacial entre las abejas y las plantas que polinizan [11].
- **Especies invasoras:** Introducción de depredadores y competidores que alteran las redes de polinización establecidas.

2.1.3. Estructura Social de la Colmena y Rol de la Reina

Las abejas melíferas son insectos eusociales altamente organizados. Una colonia típica en verano puede contener entre 30,000 y 80,000 individuos, reduciendo a tan solo 5,000 en invierno. La colmena está compuesta por tres castas principales: la reina (única hembra fértil), miles de obreras (hembras estériles) y cientos de zánganos (machos) durante la temporada reproductiva [4].

La reina es el eje central de la colonia, siendo la única capaz de poner huevos fertilizados. **Su presencia es crítica para la supervivencia a largo plazo de la colmena**, ya que

regula la cohesión social mediante la producción de feromonas específicas, particularmente la feromona mandibular de la reina (QMP) [9]. La **pérdida de la reina**, conocida como estado de *queenlessness*, resulta en un declive gradual de la población que **eventualmente lleva a la extinción de la colonia** si no se toman medidas correctivas oportunas [9].

Las **obreras** realizan todas las tareas necesarias para el mantenimiento de la colonia, incluyendo construcción de panales, recolección de néctar y polen (forrajeo), alimentación de larvas, regulación de temperatura, defensa y limpieza. Sus funciones varían según su edad en un proceso conocido como *polietismo temporal* [4].

Los **zánganos** tienen como función primordial la fecundación de reinas vírgenes. Aunque no participan en tareas laborales dentro de la colmena, pueden contribuir a la termorregulación del nido [4].

2.1.4. Necesidad de Sistemas de Monitoreo No Invasivos

El monitoreo tradicional de colmenas mediante inspección visual directa es laborioso, requiere experiencia, puede ser estresante para las abejas y potencialmente disemina enfermedades y parásitos entre colonias. Además, existe el riesgo de muerte accidental de la reina durante la inspección [9]. Por estas razones, existe una demanda creciente por sistemas de monitoreo remoto, no invasivo y continuo que permitan a los apicultores detectar tempranamente problemas

en sus colmenas.

2.2. Monitoreo Acústico de Colmenas: Estado del Arte

2.2.1. Fundamentos del Análisis Acústico en Apicultura

Las abejas producen una variedad de señales acústicas y vibratorias que transmiten información sobre el estado de la colonia. Estas señales son generadas principalmente por el batido de alas, aunque también incluyen vibraciones torácicas transmitidas al sustrato [8]. El análisis de estas señales acústicas ofrece una modalidad de monitoreo apropiada porque:

- Es completamente no invasivo.
- Permite el monitoreo continuo y en tiempo real.
- Requiere equipamiento relativamente simple (micrófono).
- Proporciona información rica sobre eventos importantes en la colmena.

2.2.2. Comunicación Acústica en Abejas Melíferas

Estudios pioneros han documentado diversos tipos de señales acústicas producidas por las abejas [8]. La Tabla 2.1 resume las principales señales identificadas:

Cuadro 2.1: Señales acústicas y vibratorias en *Apis mellifera*

Señal	Frecuencia	Duración	Contexto/Función
<i>Queen Piping</i>	340-450 Hz	1.25 s	Producida por reina virgen recién emergida
<i>Queen Quacking</i>	< 450 Hz	< 200 ms	Respuesta de reinas no emergidas
<i>Worker Piping</i> (alas juntas)	200-2000 Hz	0.2-2 s	En enjambres, modulación en frecuencia
<i>Worker Piping</i> (alas separadas)	300-400 Hz	0.2-2 s	En colmenas, sin modulación
Danza del meneo	200-300 Hz + 13-15 Hz (cola)	Pulsos cortos	Comunicación de ubicación de recursos
Señal de detención	350 Hz (media)	0.05-0.2 s	Interrupción de reclutamiento
Señal de vibración	16-18 Hz	1-2 s	Activación general de obreras

2.2.3. Detección de Eventos mediante Análisis Acústico

Detección de un comportamiento de enjambre (*Swarming*)

Swarming es el proceso natural de reproducción de colonias mediante el cual una porción significativa de la colonia abandona la colmena original con la reina vieja para establecer una nueva colonia. Este evento representa una pérdida económica considerable para el apicultor.

Ferrari et al. (2008)[12] realizaron uno de los primeros estudios cuantitativos sobre la predicción del Swarming mediante análisis acústico. Monitorearon sonidos y temperatura en el periodo alrededor de un evento de Swarming, observando que las bandas de frecuencia dominantes se desplazan de 100-300 Hz en condiciones normales a 500-600 Hz inmediatamente antes del enjambre. Este incremento fue atribuido al batido intensivo de alas en preparación para el vuelo.

Bencsik et al. (2011) utilizaron ace-

lerómetros en lugar de micrófonos, capturando vibraciones del panal. Mediante análisis de componentes principales (PCA) y eigenespectros identificaron patrones característicos previos a Swarming.

Detección de Ausencia de Reina (*Queenlessness*)

La detección del estado *queenless* es crítica porque permite intervención temprana antes de que la colonia decline irreversiblemente.

Boys (1999) reportó observaciones empíricas de apicultores experimentados, notando que el sonido “warble” característico entre 225-285 Hz disminuye en amplitud en colmenas sin reina, siendo reemplazado por un sonido “quejumbroso” a frecuencias ligeramente menores. Adicionalmente, el sonido general fue descrito como discordante en colmenas *queenless*, en contraste con el sonido armonioso de colmenas *queenright* (aquellos panales que si conservan a la reina).

Howard et al. (2013) [8] realizaron el primer estudio cuantitativo sistemático sobre detección de *queenlessness* mediante procesa-

miento de señales. Utilizaron:

- Grabaciones de cuatro colmenas (dos control *queenright*, dos experimentales con reina removida).
- Análisis espectral mediante Transformada de Stockwell.
- Clasificación mediante Mapas Auto-organizados (SOM).
- Análisis de Densidad Espectral de Potencia (PSD).

Aunque sus resultados fueron algo inconclusos, establecieron la metodología base y confirmaron diferencias espectrales entre ambos estados, particularmente en el rango 165-285 Hz.

Ruvinga et al. (2021, 2023) representan el avance más significativo en esta área, aplicando técnicas modernas de aprendizaje profundo. Su trabajo se desarrolló en dos fases:

Fase 1 (2021)[4]: Comparación de clasificadores usando MFCC

- **Dataset:** 480 grabaciones de 1 minuto (Aug 2012, 4 colmenas).

- **Características:** 14 MFCC (13 coeficientes + log-energía).
- **Modelos evaluados:** Regresión Logística, MLP, LSTM.
- **Resultados:** LSTM alcanzó 92 % de precisión, superando a MLP (90 %) y Regresión Logística (87 %).

Fase 2 (2023)[9]: Extensión con CNN y validación cruzada

- Incorporación de espectrogramas STFT como entrada para CNN.
- CNN alcanzó 99.31 % de precisión (training) y 99 % (validación).
- Validación en dataset independiente (Surrey, UK, verano 2020): CNN mantuvo 98.61 % de precisión.
- Confirmación de generalización del modelo a diferentes ubicaciones y temporadas.

La Tabla 2.2 resume los resultados comparativos:

Cuadro 2.2: Comparación de modelos para detección de *queenlessness*

Modelo	Características	Precisión	Recall	F1-Score
Reg. Logística	14 MFCC	87.4 %	-	-
MLP	14 MFCC	90.1 %	-	-
LSTM	14 MFCC	92.0 %	92 %	92 %
CNN	Espectrograma	99.31 %	99.31 %	99.31 %

2.2.4. Otros Enfoques de Machine Learning

Robles-Guerrero et al. (2019, 2020)[13] exploraron diferentes aproximaciones:

- **Lasso Logistic Regression** combinada con Singular Value Decomposition (SVD)

para clasificación multiclase de estados de colmena.

- Identificación de patrones sonoros específicos en colmenas sin reina.
- Enfoque en características espectrales de alta dimensión reducidas mediante SVD.

Peng et al. (2020)[14] reportaron el uso de MLP con dos capas ocultas y función de activación tangente hiperbólica, alcanzando 90 % de precisión con señales limpias, aunque el desempeño cayó a 61 % con señales ruidosas, evidenciando la importancia del preprocesamiento de señales.

2.2.5. Limitaciones de Estudios Previos y Brechas de Conocimiento

A pesar de los avances significativos, persisten limitaciones importantes:

1. **Datos limitados:** la mayoría de los estudios usan datasets pequeños de pocos días y pocas colmenas.
2. **Condiciones controladas:** pocos estudios se validan en condiciones de campo reales con ruido ambiental variable.
3. **Generalización:** falta evidencia de que modelos funcionen en diferentes:
 - Razas de abejas (*A. m. ligustica* vs *A. m. carnica*, etc.).
 - Condiciones climáticas y estaciones del año.
 - Ubicaciones geográficas.
4. **Sistemas embebidos:** pocos trabajos abordan la implementación en hardware de bajo costo para monitoreo en tiempo real.

2.3. Fundamentos de Procesamiento Digital de Señales de Audio

2.3.1. Representación Digital de Señales de Audio

Una señal de audio continua $x(t)$ se convierte en una señal digital mediante muestreo a intervalos regulares determinados por la frecuencia

de muestreo f_s . La señal digital resultante se representa como una secuencia discreta:

$$\begin{aligned} x[n] &= x(t = nT) , \\ n &= 0, 1, 2, \dots, N - 1 , \end{aligned} \quad (2.1)$$

donde $T = 1/f_s$ siendo este término el periodo de muestreo y N es el número total de muestras.

Según el **Teorema de Muestreo de Nyquist-Shannon**, para evitar aliasing (solapamiento espectral), la frecuencia de muestreo debe ser al menos el doble de la frecuencia máxima contenida en la señal:

$$f_s \geq 2f_{\max} . \quad (2.2)$$

En estudios de audio de colmenas, Ruvunga et al. [9] utilizaron $f_s = 44.1$ kHz, mientras que otros trabajos han usado 20.38 kHz, ambos valores adecuados considerando que la energía significativa en sonidos de abejas se concentra por debajo de 11 kHz [9].

2.3.2. Características de Señales de Audio de Colmenas

Las señales de audio de colmenas presentan características particulares [9]:

- **No estacionarias:** sus propiedades estadísticas varían en el tiempo.
- **Multicomponente:** contienen componentes armónicos (batido de alas), transitorios (impactos, movimientos) y estocásticos (ruido de fondo).
- **Energía concentrada:** frecuencia fundamental ~ 100 Hz, con información relevante hasta 6 kHz.
- **Variabilidad temporal:** cambios según hora del día, actividad de forrajeo y condiciones climáticas.

2.3.3. Transformada de Fourier de Tiempo Corto (STFT)

Debido a la naturaleza no estacionaria de las señales de audio, el análisis frecuencial tradicional mediante Transformada de Fourier (DFT) es insuficiente, ya que proporciona el contenido frecuencial promedio pero pierde completamente la información temporal.

La **Transformada de Fourier de Tiempo Corto (STFT)** resuelve esta limitación dividiendo la señal en segmentos cortos (ventanas) donde se puede asumir estacionariedad local, aplicando la DFT a cada segmento:

$$X[m, k] = \sum_{n=0}^{L-1} x[n+mH] \cdot w[n] \cdot e^{-j \frac{2\pi kn}{N}}, \quad (2.3)$$

donde:

- $x[n]$: señal de entrada.
- $w[n]$: función de ventana de longitud L .
- m : índice de trama ($m = 0, 1, 2, \dots$).
- H : hop size (salto entre ventanas, típicamente $H = L/2$ para 50 % de traslape).
- k : índice de frecuencia ($k = 0, 1, \dots, N - 1$).
- N : longitud de la FFT (típicamente $N \geq L$, potencia de 2 para eficiencia).

El **espectrograma** se obtiene como el módulo al cuadrado de la STFT:

$$S[m, k] = |X[m, k]|^2, \quad (2.4)$$

el cual representa la densidad de energía en el plano tiempo-frecuencia y se visualiza típicamente como una imagen donde:

- Eje horizontal: tiempo (índice de trama m).
- Eje vertical: frecuencia (índice de bin k).

- Intensidad/color: magnitud de la energía.

Segmentación y Ventaneo

La señal se divide en **tramas** (*frames*) de duración típica entre 20-40 ms, con un traslape (*overlap*) entre tramas consecutivas. Esta duración se selecciona porque:

- Es suficientemente corta para capturar variaciones rápidas.
- Es suficientemente larga para obtener resolución frecuencial adecuada.
- Minimiza efectos de no-estacionariedad local.

Para una frecuencia de muestreo f_s y duración de trama D (en segundos), el número de muestras por ventana es:

$$L = \lfloor D \cdot f_s \rfloor.$$

Ejemplo: Para $f_s = 44100$ Hz y $D = 25$ ms:

$$L = \lfloor 0.025 \times 44100 \rfloor = 1102.$$

En la práctica, se suele ajustar L a la potencia de 2 más cercana para eficiencia de la FFT, por ejemplo $L = 1024$.

El **hop size** H determina el desplazamiento entre tramas consecutivas. Para un traslape del 50 %:

$$H = \frac{L}{2}.$$

Esto significa que cada trama comparte la mitad de sus muestras con las tramas adyacentes. El traslape es particularmente importante cuando se usan ventanas que atenúan los extremos, garantizando que cada muestra contribuya aproximadamente igual al análisis global.

El número total de tramas M para una señal de longitud N_{total} es:

$$M = \left\lfloor \frac{N_{\text{total}} - L}{H} \right\rfloor + 1.$$

Funciones de Ventana

Se aplica una función de ventana $w[n]$ a cada trama para reducir el efecto de Gibbs (oscilaciones espectrales espurias) causado por la discontinuidad en los bordes de segmentos rectangulares. Las ventanas más comunes son:

Ventana de Hamming:

$$w[n] = 0.53836 - 0.46164 \cos\left(\frac{2\pi n}{L-1}\right),$$

$$0 \leq n \leq L-1.$$

Frecuentemente aproximada como:

$$w[n] \approx 0.54 - 0.46 \cos\left(\frac{2\pi n}{L-1}\right),$$

$$0 \leq n \leq L-1.$$

Ventana de Hanning (Hann):

$$w[n] = 0.5 - 0.5 \cos\left(\frac{2\pi n}{L-1}\right),$$

$$0 \leq n \leq L-1.$$

Ambas se pueden expresar en forma generalizada como:

$$w[n] = (1 - \omega) - \omega \cos\left(\frac{2\pi n}{L-1}\right),$$

$$0 \leq n \leq L-1.$$

donde $\omega = 0.5$ para Hanning y $\omega \approx 0.46$ para Hamming.

La ventana de Hamming ofrece mejor supresión de lóbulos laterales (~ 43 dB) comparada con Hanning (~ 31 dB), lo que reduce interferencia espectral entre componentes frecuenciales adyacentes.

2.3.4. Transformada Rápida de Fourier (FFT)

La FFT es un algoritmo eficiente para calcular la Transformada Discreta de Fourier (DFT). Para una trama ventaneada $x_w[n] = x[n] \cdot w[n]$:

$$X[k] = \sum_{n=0}^{N-1} x_w[n] \cdot e^{-j\frac{2\pi kn}{N}},$$

$$k = 0, 1, \dots, N-1.$$

La complejidad computacional de la FFT es $O(N \log N)$ comparada con $O(N^2)$ de la DFT directa, lo que la hace práctica para procesamiento en tiempo real.

Los bins frecuenciales resultantes tienen una resolución frecuencial de:

$$\Delta f = \frac{f_s}{N}.$$

Y cada bin k corresponde a una frecuencia física de:

$$f_k = \frac{k \cdot f_s}{N},$$

$$k = 0, 1, \dots, \left\lfloor \frac{N}{2} \right\rfloor.$$

El rango útil de frecuencias (espectro unilateral) va de 0 Hz hasta la frecuencia de Nyquist $f_s/2$.

Ejemplo: Con $f_s = 44100$ Hz y $N = 1024$:

$$\Delta f = \frac{44100}{1024} \approx 43.07 \text{ Hz},$$

proporcionando bins frecuenciales de aproximadamente 43 Hz de ancho, con el espectro útil abarcando de 0 Hz a 22050 Hz (frecuencia de Nyquist).

2.4. Coeficientes Cepstrales en Frecuencia Mel (MFCC)

2.4.1. Motivación y Fundamento Perceptual

Los Coeficientes Cepstrales en Frecuencia Mel (MFCC) constituyen una representación

compacta y robusta del contenido espectral de señales de audio, inspirada en el sistema auditivo humano [9]. Su diseño se basa en dos observaciones fundamentales de la psicoacústica:

1. **Percepción no lineal de frecuencias:** el oído humano no percibe frecuencias de manera lineal. Es más sensible a cambios en bajas frecuencias que en altas.
2. **Enmascaramiento frecuencial:** componentes espectrales cercanos se enmascaran mutuamente, haciendo innecesaria una resolución espectral uniforme muy alta.

Los MFCC han demostrado excelencia en múltiples dominios:

- Reconocimiento automático de voz (ASR) [9].
- Clasificación de sonidos ambientales.
- Clasificación de géneros musicales.
- Reconocimiento de sonidos cardíacos.
- **Clasificación de estados de colmenas** [4, 9].

2.4.2. La Escala Mel

La **escala Mel** es una escala perceptual de tonos (*pitch*) propuesta por Stevens et al. [15], definida de tal forma que intervalos iguales en la escala Mel son percibidos como igualmente espaciados por oyentes humanos.

La relación entre frecuencia física f (Hz) y frecuencia percibida m (Mel) se define como [16,

17]:

$$m = 2595 \log_{10} \left(1 + \frac{f}{700} \right) . \quad (2.5)$$

Alternativamente, con base natural:

$$m = 1127 \ln \left(1 + \frac{f}{700} \right) . \quad (2.6)$$

La transformación inversa es:

$$f = 700 (10^{m/2595} - 1) . \quad (2.7)$$

Por diseño, 1000 Hz corresponde a aproximadamente 1000 Mel, sirviendo como punto de referencia. La escala comprime progresivamente las frecuencias altas, modelando la mayor sensibilidad del oído humano a diferencias de tono en el rango de bajas frecuencias.

Esta relación aproxima el comportamiento del sistema auditivo humano:

- Alrededor de 1000 Hz: punto de referencia donde $m \approx f$ (1000 Hz $\approx$ 1000 Mel).
- Para frecuencias bajas: la percepción del tono es aproximadamente lineal con la frecuencia.
- Para frecuencias altas: la percepción del tono sigue una relación logarítmica.

La Figura 2.1 ilustra esta relación no lineal entre la Frecuencia Física y la Escala Mel.

2.4.3. Proceso de Extracción de MFCC

La extracción de MFCC es un proceso de 7 etapas secuenciales [9], ilustrado en la Figura 2.2.

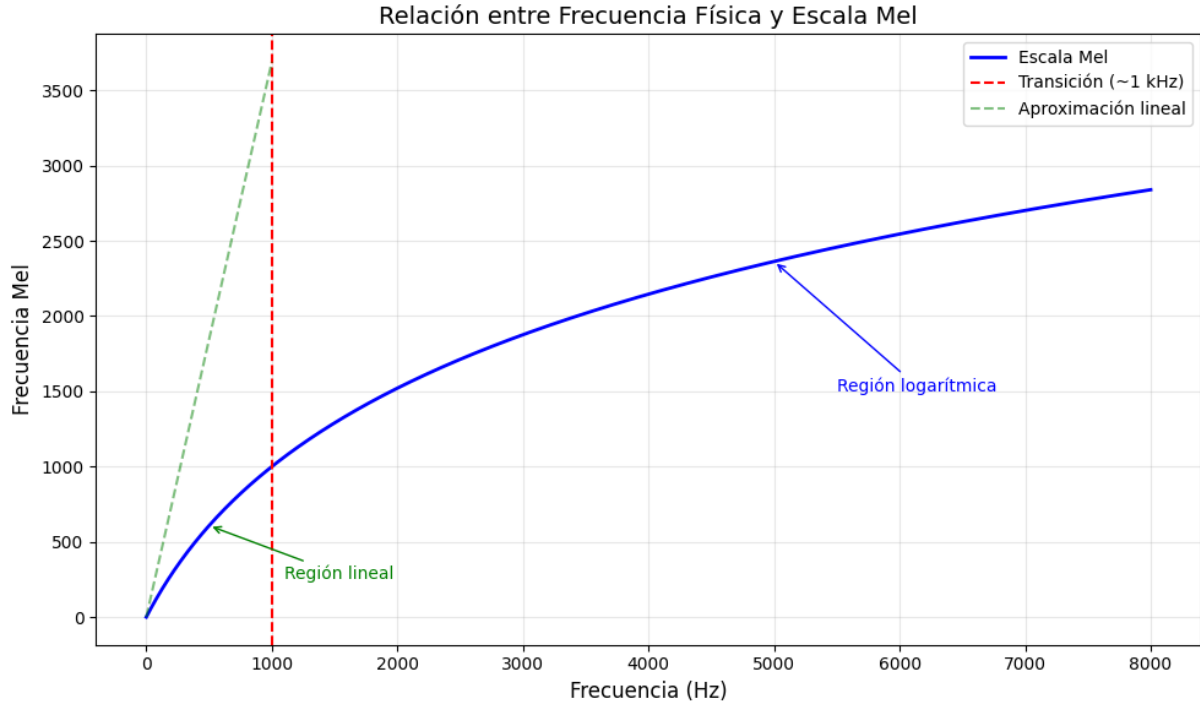


Figura 2.1: Relación entre frecuencia física (Hz) y escala Mel. Nótese la transición de comportamiento lineal a logarítmico alrededor de 1 kHz.

Etapla 1: Pre-énfasis

El **filtro de pre-énfasis** es un filtro pasa-altos de primer orden que amplifica las componentes de alta frecuencia, compensando la caída natural de energía en altas frecuencias que ocurre durante la generación y grabación de audio.

Se implementa como un filtro FIR de primer orden:

$$y[n] = x[n] - \alpha \cdot x[n - 1] , \quad (2.8)$$

donde α es el coeficiente de pre-énfasis, típicamente $0.95 \leq \alpha \leq 0.97$.

En el dominio z :

$$H_{pre}(z) = 1 - \alpha z^{-1} . \quad (2.9)$$

En estudios de audio de colmenas, este filtro es particularmente importante porque [9]:

- Información relevante existe en bandas de 3-6 kHz.

- Estas frecuencias tienden a atenuarse durante grabación.
- El pre-énfasis mejora la discriminación espectral.

Etapla 2: Segmentación (*Framing*) y Ventaneo

Ya descrita en la sección 2.3.3, la señal se segmenta en tramas con traslape. Para MFCC, los parámetros típicos son [9]:

- Duración de trama: 20-40 ms (25 ms es común).
- Traslape: 50 % (12.5 ms si trama = 25 ms).
- Ventana: Hamming o Hanning.

Para cada trama i :

$$\begin{aligned} s_i[n] &= x[n + iH] \cdot w[n] , \\ n &= 0, \dots, N - 1 , \end{aligned} \quad (2.10)$$

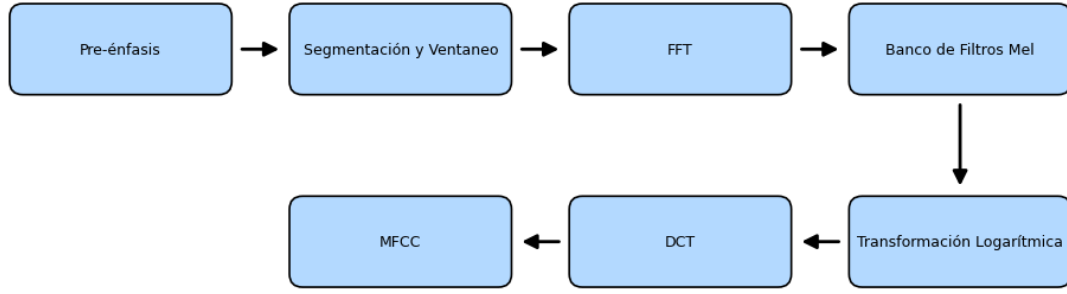


Figura 2.2: Proceso de extracción de MFCC en 7 etapas.

donde H es el salto entre tramas (*hop size*).

Etapa 3: Transformada Rápida de Fourier (FFT)

Se aplica la FFT a cada trama ventaneada para obtener el espectro de magnitud:

$$S_i[k] = \left| \sum_{n=0}^{N-1} s_i[n] e^{-j \frac{2\pi kn}{N}} \right|^2, \quad (2.11)$$

$$k = 0, \dots, \frac{N}{2}.$$

Solo se conservan los primeros $N/2+1$ bins (frecuencias positivas) debido a la simetría de la FFT para señales reales.

Etapa 4: Banco de Filtros Mel

El **banco de filtros Mel** es el componente que implementa la escala perceptual. Consiste en un conjunto de M filtros triangulares solapados, distribuidos en escala Mel, que cubren el rango de frecuencias de interés.

Construcción del banco de filtros:

1. **Definir rango:** Frecuencias mínima f_{min} y máxima f_{max} (ej: 0 Hz a $f_s/2$).

2. **Convertir a escala Mel:**

$$m_{min} = 2595 \log_{10} \left(1 + \frac{f_{min}}{700} \right),$$

$$m_{max} = 2595 \log_{10} \left(1 + \frac{f_{max}}{700} \right).$$

3. **Dividir uniformemente en Mel:** Crear $M+2$ puntos equiespaciados

$$m[b] = m_{min} + \frac{b}{M+1} (m_{max} - m_{min}),$$

$$b = 0, \dots, M+1.$$

4. **Convertir de vuelta a Hz:**

$$f[b] = 700 (10^{m[b]/2595} - 1).$$

5. **Mapear a bins de FFT:**

$$\text{bin}[b] = \left\lfloor \frac{(N+1) \cdot f[b]}{f_s} \right\rfloor.$$

6. **Construir filtros triangulares:** Para cada filtro m ($m = 1, \dots, M$):

$$H_m[k] = \begin{cases} 0 & k < \text{bin}[m-1] \\ \frac{k - \text{bin}[m-1]}{\text{bin}[m] - \text{bin}[m-1]} & \text{bin}[m-1] \leq k < \text{bin}[m] \\ \frac{\text{bin}[m+1] - k}{\text{bin}[m+1] - \text{bin}[m]} & \text{bin}[m] \leq k < \text{bin}[m+1] \\ 0 & k \geq \text{bin}[m+1] \end{cases}$$

Los filtros se aplican al espectro de potencia:

$$E_m = \sum_{k=0}^{N/2} S[k] \cdot H_m[k],$$

$$m = 1, \dots, M.$$

Parámetros típicos:

- $M = 20 - 40$ filtros (26-40 es común).
- $f_{\min} = 0$ Hz o 20 Hz.
- $f_{\max} = f_s/2$ o valor menor si se conoce BW del contenido.

Etap 5: Logaritmo

Se aplica logaritmo para:

1. Comprimir el rango dinámico.
2. Aproximar la percepción logarítmica de sonoridad humana.
3. Convertir productos espectrales en sumas (separación de fuente/filtro).

$$\log E_m = \log \left(\sum_k S[k] \cdot H_m[k] \right),$$

$$m = 1, \dots, M.$$

Típicamente se usa $\log_{10}$ o $\ln$. Para evitar $\log(0)$:

$$\log E_m = \log(\max(E_m, \epsilon)).$$

donde ϵ es un valor pequeño (ej: 10^{-10}).

Etap 6: Transformada Discreta del Coseno (DCT)

La **DCT** decorrela las energías de los bancos de filtros Mel, que tienden a estar correlacionadas debido al traslape entre filtros.

La DCT de tipo II se define como:

$$c[n] = \sum_{m=1}^M \log E_m \cdot \cos \left[\frac{\pi n(m - 0.5)}{M} \right],$$

$$n = 0, \dots, M - 1.$$

Los coeficientes resultantes $c[n]$ son los **MFCC**. Típicamente se conservan solo los primeros C coeficientes (usualmente $C = 12$ o 13):

$$\text{MFCC} = [c_0, c_1, c_2, \dots, c_{C-1}].$$

Interpretación física:

- c_0 : Relacionado con energía promedio (a veces se descarta o reemplaza por log-energía).
- c_1, c_2 : Capturan envolvente espectral gruesa.
- $c_3, \dots, c_{12}$: Detalles espectrales más finos.
- $c_{>12}$: Usualmente desechados (información menos relevante, más ruido).

Eta 7: Características Dinámicas (Δ y Δ^2)

Los MFCC estáticos capturan información espectral instantánea pero ignoran la dinámica temporal. Las **derivadas temporales** capturan estos cambios:

Delta (velocidad): primera derivada, captura tasa de cambio

$$\Delta c[n, t] = \frac{\sum_{\theta=1}^{\Theta} \theta (c[n, t + \theta] - c[n, t - \theta])}{2 \sum_{\theta=1}^{\Theta} \theta^2}.$$

Delta-Delta (aceleración): segunda derivada

$$\Delta^2 c[n, t] = \frac{\sum_{\theta=1}^{\Theta} \theta (\Delta c[n, t + \theta] - \Delta c[n, t - \theta])}{2 \sum_{\theta=1}^{\Theta} \theta^2},$$

donde Θ es la ventana de cálculo (típicamente $\Theta = 2$).

El **vector de características completo** para cada trama es:

$$\mathbf{f}_t = [\text{MFCC}_t, \Delta \text{MFCC}_t, \Delta^2 \text{MFCC}_t].$$

Si se usan 13 MFCC, el vector tiene dimensión $13 \times 3 = 39$ características.

2.4.4. Configuraciones de MFCC en Estudios de Colmenas

Ruvunga et al. [4, 9] evaluaron 4 configuraciones:

Resultados clave:

- Dataset 1 (14 características) dio **mejores resultados**.
- c_0 (relacionado con energía DC) y log-energía son informativos.
- Dataset 4 (12 características, sin c_0 ni log-energía) dio peores resultados.

Esto sugiere que tanto la información de energía global como la estructura armónica completa son relevantes para discriminar estados de colmena.

2.4.5. Implementación Práctica

Librerías ampliamente usadas para MFCC:

- **Python**: `librosa.feature.mfcc()`, `python_speech_features`.
- **MATLAB**: `mfcc()` en Audio Toolbox.
- **C++**: Bibliotecas como `Essentia`, `Aquila`.

Parámetros típicos para audio de colmenas basados en tu trabajo:

```

1 import librosa
2
3 mfcc = librosa.feature.mfcc(
4     y=audio_signal,
5     sr=22050,           # Frecuencia de muestreo
6     n_mfcc=13,          # Numero de coeficientes
7     n_fft=2048,         # Tamano de FFT
8     hop_length=512,     # Salto entre tramas
9     n_mels=40,           # Numero de filtros Mel
10    fmin=0,              # Frecuencia minima
11    fmax=8000            # Frecuencia maxima

```

2.5. Aprendizaje Profundo para Clasificación de Audio

2.5.1. Conceptos Fundamentales de Redes Neuronales

Las **redes neuronales artificiales (ANN)** son modelos computacionales inspirados en el sistema nervioso biológico, compuestos por unidades interconectadas (neuronas artificiales) organizadas en capas.

Perceptrón y Arquitectura Básica

Una neurona artificial implementa:

$$y = \phi \left(\sum_{i=1}^n w_i x_i + b \right) = \phi(\mathbf{w}^T \mathbf{x} + b) ,$$

donde:

- $\mathbf{x} = [x_1, \dots, x_n]^T$: vector de entrada.
- $\mathbf{w} = [w_1, \dots, w_n]^T$: pesos sinápticos.
- b : sesgo (*bias*).
- $\phi(\cdot)$: función de activación no lineal.

Funciones de Activación

Sigmoide (Logística):

$$\sigma(z) = \frac{1}{1 + e^{-z}} \in (0, 1) ,$$

usada en capas de salida para clasificación binaria.

Tangente Hiperbólica:

$$\tanh(z) = \frac{e^{2z} - 1}{e^{2z} + 1} \in (-1, 1) ,$$

preferida sobre sigmoide en capas ocultas por estar centrada en cero.

ReLU (Rectified Linear Unit):

$$\text{ReLU}(z) = \max(0, z) ,$$

actualmente la más popular por:

- Eficiencia computacional.
- Mitigación del problema de gradientes desvanecientes.
- Inducción de escasez (sparsity).

Softmax (para clasificación multiclase):

$$\text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} ,$$

donde K es el número de clases.

Entrenamiento: Retropropagación y Optimización

El entrenamiento consiste en ajustar $\mathbf{w}$ y b para minimizar una función de pérdida $\mathcal{L}$.

Función de pérdida para clasificación binaria (Entropía Cruzada):

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] .$$

El algoritmo de **retropropagación** calcula gradientes:

$$\frac{\partial \mathcal{L}}{\partial w_{ij}} = \frac{\partial \mathcal{L}}{\partial y} \cdot \frac{\partial y}{\partial z} \cdot \frac{\partial z}{\partial w_{ij}} ,$$

y los parámetros se actualizan mediante **descenso de gradiente**:

$$w_{ij}^{(t+1)} = w_{ij}^{(t)} - \eta \frac{\partial \mathcal{L}}{\partial w_{ij}} ,$$

donde η es la tasa de aprendizaje.

2.5.2. Perceptrón Multicapa (MLP)

Un MLP es una red feedforward con al menos una capa oculta. Para clasificación binaria de estados de colmena:

- **Capa de entrada:** n neuronas (dimensión de MFCC, ej: 14).
- **Capas ocultas:** L capas con $h_1, \dots, h_L$ neuronas cada una.
- **Capa de salida:** 1 neurona con activación sigmoide.

Ruvinga et al. [9] reportaron:

- MLP con 14 MFCC de entrada.
- Función de activación: tangente hiperbólica en capas ocultas.
- Precisión: 90.1 % (Dataset 1).

2.5.3. Regresión Logística

Puede verse como un MLP sin capas ocultas (modelo lineal + sigmoide):

$$P(y = 1|\mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x} + b) .$$

Ruvinga et al.: Precisión 87.4 % con 14 MFCC.

Limitación: Asume separabilidad lineal en el espacio de características.

2.5.4. Redes Neuronales Recurrentes (RNN) y LSTM

Limitaciones de MLP para Secuencias

Los MLP procesan cada trama de forma independiente, ignorando:

- Dependencias temporales entre tramas.
- Contexto histórico de la señal.
- Patrones secuenciales.

Para audio, esto es subóptimo porque el significado depende de la evolución temporal.

Redes Neuronales Recurrentes (RNN)

Las RNN mantienen un **estado oculto** que se actualiza en cada paso temporal:

$$a_t = \tanh(W_{hh}a_{t-1} + W_{hx}x_t + b_h) ,$$

$$y_t = W_{hy}a_t + b_y .$$

Ventajas:

- Memoria de eventos pasados.
- Parámetros compartidos en el tiempo (generalización).

Problema crítico: Desvanecimiento/explosión de gradientes.

Durante retropropagación a través del tiempo (BPTT), los gradientes se calculan como productos de derivadas:

$$\frac{\partial \mathcal{L}}{\partial a_1} = \frac{\partial \mathcal{L}}{\partial a_T} \prod_{t=2}^T \frac{\partial a_t}{\partial a_{t-1}} .$$

Si $\|\frac{\partial a_t}{\partial a_{t-1}}\| < 1$: gradientes desvanecen exponencialmente.

Si $\|\frac{\partial a_t}{\partial a_{t-1}}\| > 1$: gradientes explotan.

Consecuencia: Incapacidad de aprender dependencias a largo plazo.

Long Short-Term Memory (LSTM)

Las LSTM [18] resuelven el problema de gradientes mediante una arquitectura especializada que fuerza flujo constante de error.

Arquitectura de la Célula LSTM:

- **Estado de celda** c_t : "memoria" de largo plazo.
- **Estado oculto** a_t : salida de corto plazo.
- **Tres compuertas** que controlan flujo de información:

- Compuerta de olvido f_t .
- Compuerta de entrada i_t .
- Compuerta de salida o_t .

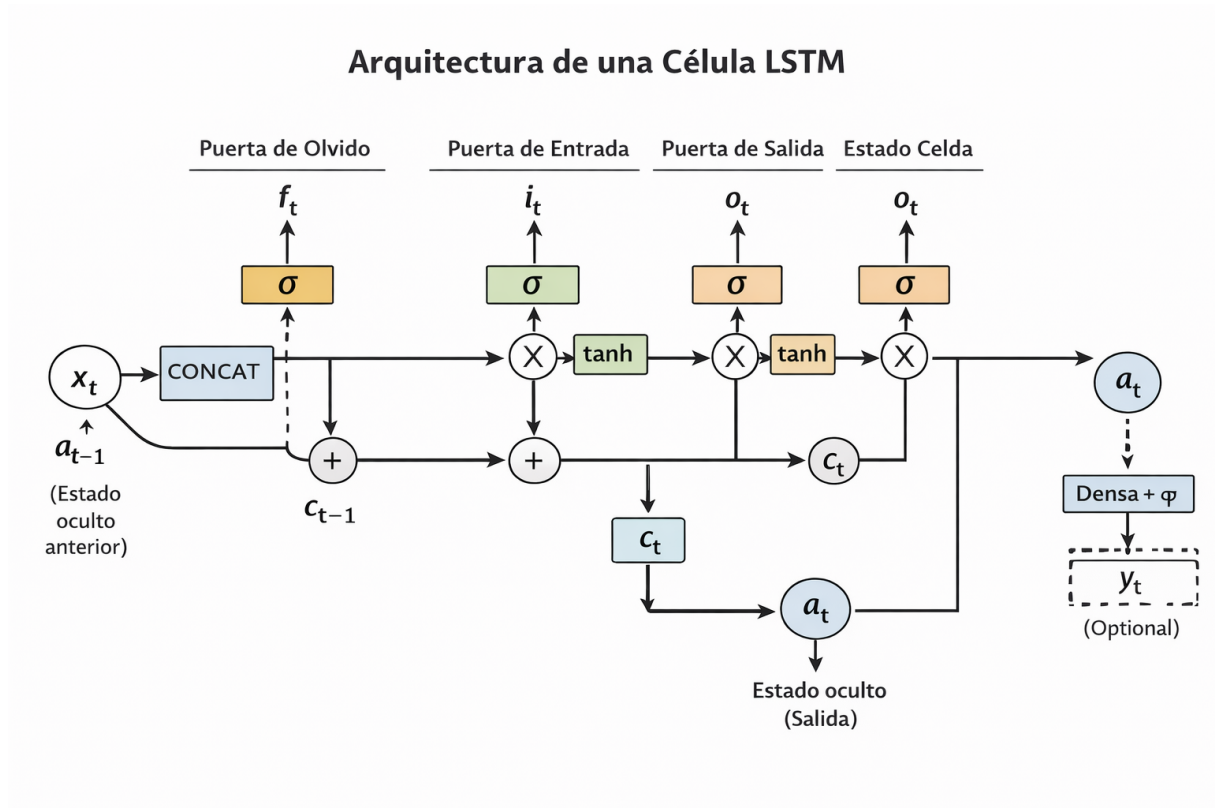


Figura 2.3: Arquitectura de una célula LSTM mostrando las tres compuertas y los estados de celda.

Ecuaciones de la LSTM:

$$\begin{aligned}
 f_t &= \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) && \text{(compuerta de olvido).} \\
 i_t &= \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) && \text{(compuerta de entrada).} \\
 \tilde{c}_t &= \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) && \text{(candidato a estado de celda).} \\
 c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t && \text{(actualización del estado).} \\
 o_t &= \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) && \text{(compuerta de salida).} \\
 a_t &= o_t \odot \tanh(c_t) && \text{(estado oculto de salida).}
 \end{aligned}$$

donde:

- σ : función sigmoide (salidas en $[0, 1]$).
- $\odot$: producto de Hadamard (elemento a elemento).

Interpretación de las Compuertas:

1. **Compuerta de olvido** ($f_t \in [0, 1]$):

- Decide qué información del estado previo c_{t-1} descartar.
- $f_t \approx 0$: olvida completamente.
- $f_t \approx 1$: retiene completamente.

2. Compuerta de entrada ($i_t \in [0, 1]$):

- Decide qué nueva información almacenar en c_t .
- Filtra el candidato $\tilde{c}_t$.
- $i_t \approx 0$: ignora nueva información.
- $i_t \approx 1$: incorpora completamente.

3. Compuerta de salida ($o_t \in [0, 1]$):

- Decide qué partes del estado de celda c_t exponer como salida a_t .

Flujo de Gradientes: la clave del éxito de LSTM está en la actualización del estado de celda:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t .$$

El gradiente fluye como:

$$\frac{\partial c_t}{\partial c_{t-1}} = f_t .$$

Si $f_t \approx 1$, el gradiente fluye sin atenuación (evita desvanecimiento).

LSTM para Clasificación de Estados de Colmena

Ruvinga et al. [4, 9] implementaron:

- **Entrada:** secuencia de vectores MFCC $\mathbf{X} = [x_1, x_2, \dots, x_T]$ donde $x_t \in \mathbb{R}^{14}$.
- **Arquitectura:**
 - 1 capa LSTM con 100 unidades.
 - Capa densa (fully connected) con 1 neurona + sigmoide.
- **Función de pérdida:** entropía cruzada binaria.

- **Optimizador:** ADAM (Adaptive Moment Estimation).

- Combina momentum y RMSProp.
- Tasas de aprendizaje adaptativas por parámetro.
- Eficiente computacionalmente.

■ Resultados:

- Precisión: 92 % (validación).
- F1-Score: 92 %.
- Superó a MLP (90 %) y Regresión Logística (87 %).

Ventajas de LSTM sobre MLP:

- Captura patrones temporales en la evolución del sonido.
- Detecta transiciones entre estados acústicos.
- Más robusto a variaciones temporales naturales.

2.5.5. Redes Neuronales Convolucionales (CNN)

Motivación para Audio

Mientras LSTM opera sobre secuencias de vectores (MFCC), las CNN procesan representaciones bidimensionales como espectrogramas, tratándolos como “imágenes” tiempo-frecuencia.

Ventajas de CNN:

- Detección automática de características jerárquicas.
- Invarianza a traslaciones temporales y frecuenciales.
- Uso eficiente de parámetros (compartición de pesos).
- Capacidad de detectar patrones locales y globales.

Arquitectura de CNN

Una CNN típica consta de:

1. **Capas Convolucionales:** Detectan características locales.
2. **Capas de Activación:** Introducen no linealidad (ReLU).
3. **Capas de Pooling:** Reducen dimensionalidad espacial.
4. **Capas Densas (Fully Connected):** Clasificación final.

Operación de Convolución 2D: para una entrada I (espectrograma) y un kernel K de tamaño $m \times n$,

$$(I * K)(i, j) = \sum_{p=0}^{m-1} \sum_{q=0}^{n-1} I(i+p, j+q) \cdot K(p, q) .$$

Cada filtro produce un **mapa de características** (feature map).

Capa Convolutiva:

Con C filtros, hiperparámetros:

- Número de filtros C (ej: 32, 64, 128).
- Tamaño de kernel $f \times f$ (ej: 3×3 , 5×5).
- Stride s (desplazamiento del kernel).
- Padding p (relleno de bordes).

Dimensiones de salida:

$$w_{out} = \left\lfloor \frac{w_{in} - f + 2p}{s} \right\rfloor + 1 ,$$

$$a_{out} = \left\lfloor \frac{a_{in} - f + 2p}{s} \right\rfloor + 1 .$$

Capa de Pooling: reduce dimensión espacial, reteniendo información relevante.

Max Pooling: Toma el valor máximo en cada región

$$y_{ij} = \max_{(p,q) \in R_{ij}} x_{pq} .$$

Average Pooling: Toma el promedio

$$y_{ij} = \frac{1}{|R_{ij}|} \sum_{(p,q) \in R_{ij}} x_{pq} .$$

Tamaño típico: 2×2 con stride 2 (reduce dimensión a la mitad).

CNN para Clasificación de Colmenas

Ruvunga et al. [9] implementaron:

- **Entrada:** Espectrogramas STFT de 64×64 píxeles.
- **Arquitectura:**
 - Capa Conv2D: 64 filtros, kernel 3×3 , ReLU.
 - MaxPooling: 2×2 .
 - Capa Conv2D: 128 filtros, kernel 3×3 , ReLU.
 - MaxPooling: 2×2 .
 - Flatten (aplanar).
 - Capa densa: N neuronas, ReLU.
 - Capa de salida: 1 neurona, sigmoide.
- **Preprocesamiento:**
 - Segmentos de audio de 5 segundos.
 - Espectrogramas calculados con STFT.
 - Normalización de píxeles.
- **Resultados:**
 - **Precisión (training):** 99.12 %.
 - **Precisión (validación):** 99.31 %.
 - **Precisión (test, dataset independiente):** 99 %.
 - **Recall:** 99.31 %.

- **F1-Score:** 99.31 %.

Superioridad de CNN:

- Captura patrones espacio-temporales en espectrogramas.
- Detecta texturas acústicas características.
- Aprende representaciones jerárquicas automáticamente.
- Menor sensibilidad a variaciones locales (pooling).

2.5.6. Comparación de Arquitecturas

Lecciones clave:

1. Modelado temporal (LSTM) ¿Estático (MLP, LR).
2. Representación rica (espectrograma) ¿Compacta (MFCC).
3. Aprendizaje jerárquico (CNN) muy efectivo para patrones complejos.
4. Trade-off: Precisión vs. complejidad computacional.

2.6. Síntesis y Justificación Metodológica

Este capítulo ha establecido los fundamentos teóricos para la detección automática del estado de *queenlessness* en colmenas mediante análisis acústico y aprendizaje profundo. Los principales hallazgos de la revisión son:

2.6.1. Contexto y Motivación

1. Las abejas melíferas son polinizadores críticos, contribuyendo USD \$235-577 mil millones anuales a la agricultura global.

2. El declive poblacional documentado en múltiples regiones amenaza la seguridad alimentaria.
3. La reina es esencial para la supervivencia de la colonia; su pérdida lleva a extinción si no se detecta tempranamente.
4. El monitoreo tradicional es invasivo, laborioso y riesgoso.

2.6.2. Estado del Arte en Monitoreo Acústico

1. Las abejas producen señales acústicas informativas sobre el estado de la colonia.
2. Estudios previos han identificado características espectrales distintivas de *queenlessness* (especialmente 165-285 Hz).
3. Técnicas de machine learning han mostrado precisiones de 87-92 % usando MFCC.
4. Deep learning (CNN) ha alcanzado hasta 99 % de precisión usando espectrogramas.
5. Persisten brechas en generalización, robustez al ruido, e implementación en sistemas embebidos.

2.6.3. Fundamentos Técnicos

1. MFCC proporcionan representación compacta y robusta del contenido espectral, capturando información perceptualmente relevante.
2. LSTM capturan dependencias temporales a largo plazo, superando a modelos estáticos.
3. CNN procesan espectrogramas como imágenes tiempo-frecuencia, aprendiendo características jerárquicas automáticamente.

4. La elección de características (MFCC vs. espectrograma) y arquitectura (LSTM vs. CNN) implica trade-offs entre precisión, interpretabilidad y costo computacional.

2.6.4. Justificación de la Metodología Propuesta

Basándose en esta revisión, el presente estudio adoptará:

- **MFCC como características primarias:** Por su compacidad, robustez y éxito demostrado en la literatura.
- **Evaluación de múltiples arquitecturas:** LSTM y potencialmente CNN para comparación exhaustiva.

- **Énfasis en preprocesamiento:** Segmentación, normalización, y posible denoising para mejorar robustez.
- **Validación rigurosa:** División train/validation/test con datos de múltiples colmenas y condiciones cuando sea posible.
- **Consideración de implementación:** Evaluación de viabilidad en sistemas embebidos de bajo costo.

El siguiente capítulo detallará la metodología específica implementada, incluyendo el dataset utilizado, preprocesamiento de señales, extracción de características, diseño de modelos y métricas de evaluación.

3. METODOLOGÍA

3.1. Procedimiento

Como verificación inicial, se ejecutó un programa de prueba (programa A.1) para validar la extracción de características acústicas mediante

coeficientes cepstrales en escala Mel (MFCC), asegurando dimensionalidad correcta, ausencia de valores indefinidos y consistencia temporal antes de procesar el conjunto completo.

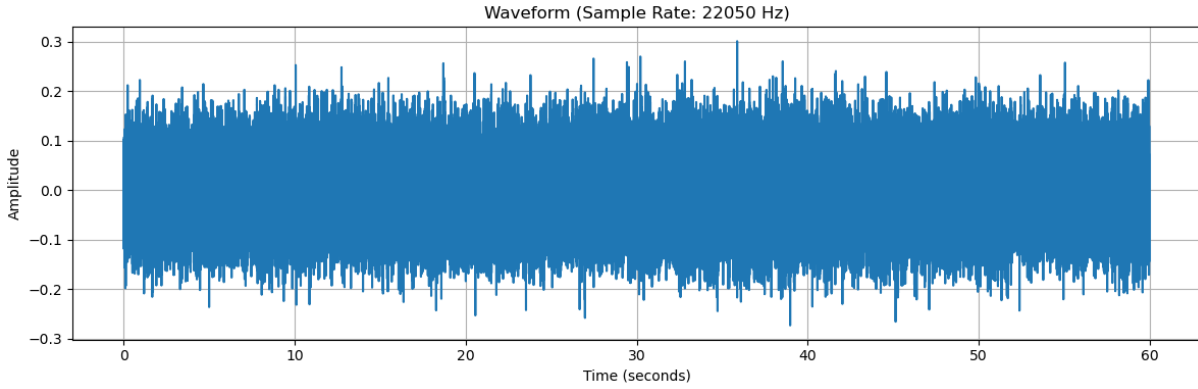


Figura 3.1: Forma de onda del archivo de audio 2022-06-05--19-39-14_2__segment2.wav.

Se trabajó con audio monofónico (de un solo canal) en formato .WAV cuya representación se muestra en la figura 3.1, con frecuencia de muestreo original de 22050 Hz y datos en formato `float32`. Para estandarizar el procesamiento, las señales fueron remuestreadas a 16 kHz durante la carga.

El flujo de procesamiento se validó por etapas en cuadernos `*.ipynb` y posteriormente se automatizó para el conjunto completo: carga, normalización, segmentación, extracción de características, generación de arreglos `*.npy` para el entrenamiento, desarrollo y aplicación del modelo.

Tras el remuestreo, se aplicó un recorte de silencios con un umbral definido como `top_db = 30`, seguido de una normalización de amplitud:

$$x_{\text{norm}}[n] = \frac{x_{\text{trim}}[n]}{A_{\text{max}}}$$

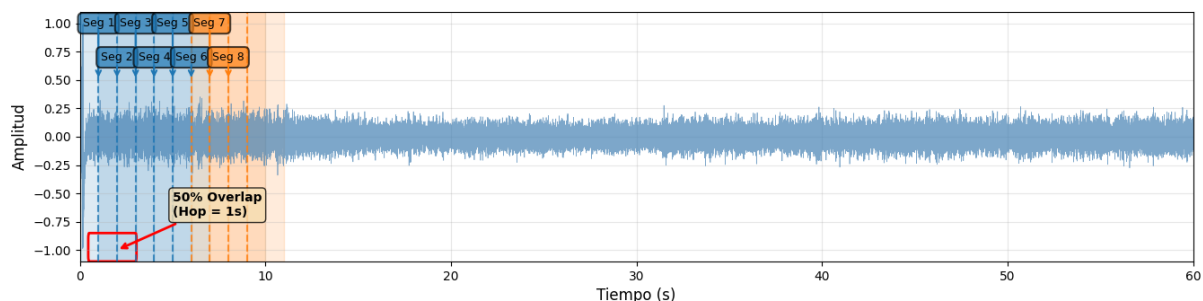
donde A_{max} es el valor absoluto máximo de la señal. Esta normalización garantiza que la señal resultante tenga valores en el rango $[-1, 1]$. Al aplicarse al archivo .WAV, reduce el efecto de diferencias globales de ganancia entre grabaciones sobre la extracción de características espectrales, permitiendo que el modelo se enfoque en patrones relevantes para la clasificación del estado de la reina.

Posteriormente, las señales fueron segmentadas en ventanas de 2 segundos con un solapamiento del 50 % (como un ejemplo gráfico, véase las figuras 3.2 (a) y (b)), generando entradas de longitud fija. Esta segmentación incrementa el número de ejemplos disponibles para el entrenamiento y, al mismo tiempo, produce entradas de duración fija para el modelo.

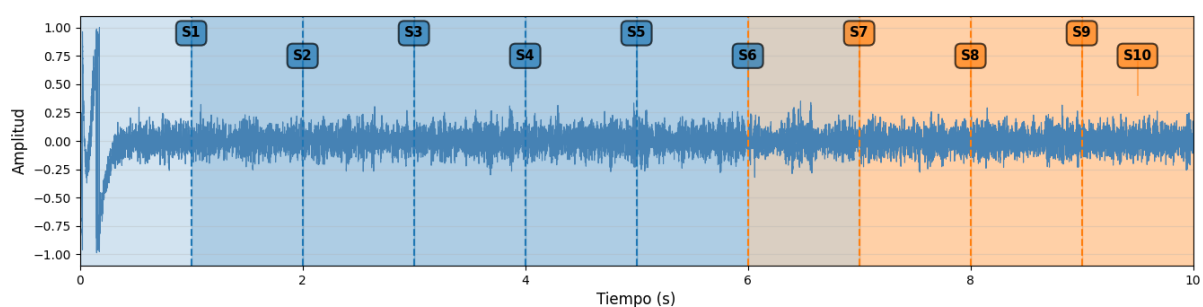
Para cada segmento se calcularon MFCC utilizando una ventana de 25 ms, un desplazamiento de 10 ms (`hop_length = 160` muestras) y $N_{\text{FFT}} = 0.025 \times 16,000 = 400$, obteniendo

una resolución espectral aproximada de 31.25 Hz. Se emplearon 40 filtros en escala Mel entre 20 Hz y 8000 Hz, lo que permite una mayor

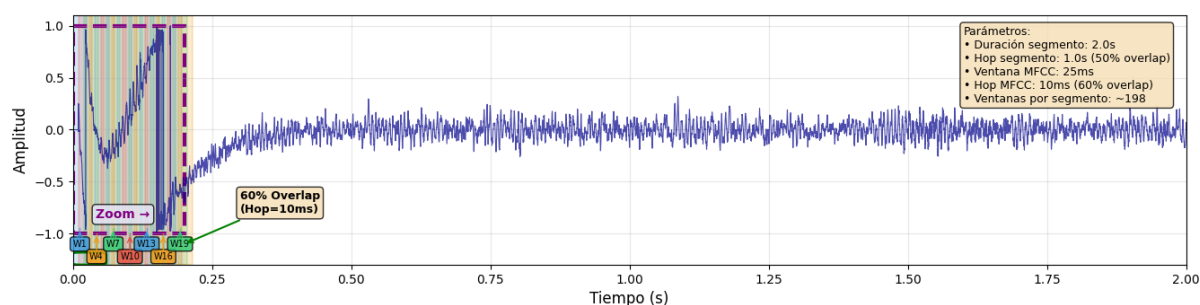
resolución en bajas frecuencias, donde se concentran patrones acústicos importantes para la clasificación del estado de la reina.



(a) Señal Original (60s) con Segmentación



(b) Zoom: Primeros 10 segundos mostrando Overlap de Segmentos



(c) Ventanas MFCC dentro del Segmento 1 (2s): 25ms con Hop 10ms

Figura 3.2: Ejemplo de segmentación de una señal de audio y ventanas MFCC.

En la figura 3.3 se muestra un acercamiento de las primeras 10 ventanas MFCC que fueron expuestas en el ejemplo de la figura 3.2 (c), donde se puede observar la representación de cada ventana de 25 ms con desplazamiento de 10 ms y solapamiento del 60 % entre ventanas consecutivas. Esta configuración permite capturar la

evolución temporal de las características espectrales, lo que es crucial para la tarea de clasificación del estado de la reina, ya que se esperaba que ciertos patrones acústicos se manifestaran de manera dinámica a lo largo del tiempo dentro de la colmena.

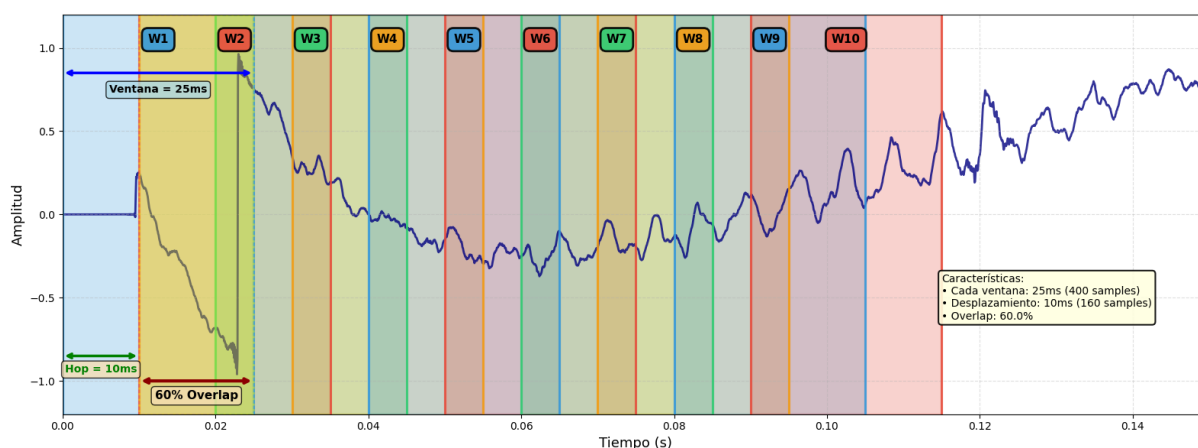


Figura 3.3: ZOOM: Primeras 10 ventanas MFCC (25 ms con Hop 10 ms) - 60 % Overlap.

Mediante la Transformada Discreta del Coseno (DCT), se calcularon 33 coeficientes MFCC y se descartó C_0 ya que representa la energía promedio del espectro y puede ser menos informativo para la clasificación especialmente en presencia de variaciones de intensidad entre grabaciones; resultando en 32 coeficientes según se muestra en la figura 3.4 (a). Además, se incluyeron deri-

vadas Δ y Δ^2 , generando una representación final de tamaño $(3 \times 32 \times T)$ por segmento, donde T es el número de cuadros temporales. Esta representación matricial (organizada en arreglos `*.npy`), permite el uso de redes convolucionales para capturar patrones espectro-temporales relevantes.

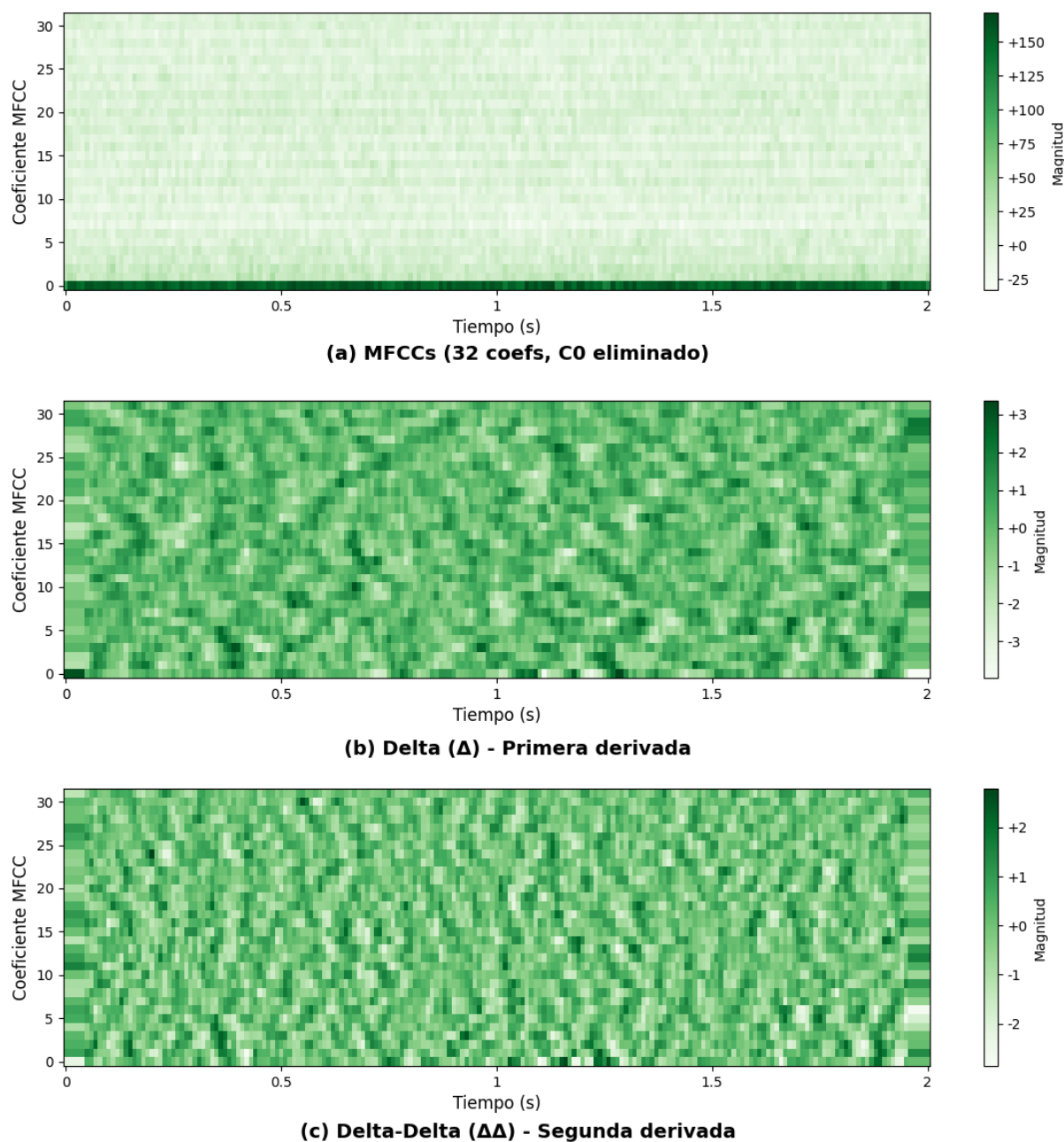


Figura 3.4: Ejemplo de estas tres capas de características para un segmento de 2 segundos.

3.2. Modelos Involucrados

Se implementaron múltiples arquitecturas de redes neuronales convolucionales (CNN) bajo un esquema base común, con el objetivo de analizar el impacto de distintas decisiones de diseño sobre el desempeño en la tarea de clasificación del estado de la reina. En particular, se evalua-

ron variaciones en la profundidad de la red, el mecanismo de reducción espacial y la función de activación.

Los modelos reciben como entrada tensores de dimensión (3, 32, 201), correspondientes a representaciones espectro-temporales derivadas de los coeficientes cepstrales en escala Mel (MFCC):

- 3 canales: MFCC estáticos, derivadas de primer orden (Δ) y derivadas de segundo orden (Δ^2).
- 32 coeficientes cepstrales (excluyendo C_0).
- 201 tramas temporales (equivalentes a segmentos de audio de 2 segundos).

Bajo este esquema, se seleccionaron dos modelos representativos por su desempeño: **CNN_1** y **CNN_2**. Ambos comparten una estructura basada en bloques convolucionales con normalización por lotes (*batch normalization*) y operaciones de *max-pooling*, pero difieren en aspectos clave de diseño arquitectónico y en la función de activación empleada.

Ambos modelos fueron entrenados bajo condiciones comparables, la función de pérdida de entropía cruzada categórica dispersa (*sparse categorical crossentropy*) y la métrica de precisión (*accuracy*). Para mitigar el desbalance de clases presente en el conjunto de entrenamiento, se emplearon pesos de clase (clase 0: 1.78, clase 1: 2.01, clase 2: 1.23, clase 3: 0.47).

3.2.1. CNN_1: Arquitectura Base con Activación ReLU

El modelo CNN_1 implementa una arquitectura convolucional de profundidad moderada compuesta por cuatro bloques convolucionales secuenciales, utilizando el optimizador Adam con una tasa de aprendizaje de 7.5×10^{-4} . Cada bloque incluye una capa convolucional con función de activación ReLU, seguida de normalización

por lotes y una operación de *max-pooling*, lo que permite una extracción progresiva de características y una reducción de la dimensionalidad espacial.

En la última etapa, incorpora una capa adicional seguida de una operación de *global average pooling*, la cual permite condensar la información espacial en un vector de características de menor dimensión, reduciendo el número de parámetros y favoreciendo la generalización del modelo. Posteriormente, las representaciones extraídas son procesadas mediante capas densas para realizar la clasificación final. Adicionalmente, para mitigar el sobreajuste, se aplicó regularización L2 (*kernel_regularizer = l2(1.5e - 4)*) en todas las capas convolucionales y en la primera capa densa. La estructura detallada del modelo CNN_1 se presenta en la tabla 3.1.

Esta arquitectura constituye una línea base que permite evaluar la capacidad de una CNN de complejidad intermedia para capturar patrones relevantes en representaciones MFCC.

El desempeño del modelo se evaluó mediante métricas de clasificación estándar, como se muestra en la tabla 3.2. CNN_1 alcanza una precisión global (*accuracy*) de 83.33 % (F1-score ponderado de 0.8356), lo que indica un desempeño adecuado en la tarea de clasificación, aunque influenciado por el desbalance entre clases presente en el conjunto de datos.

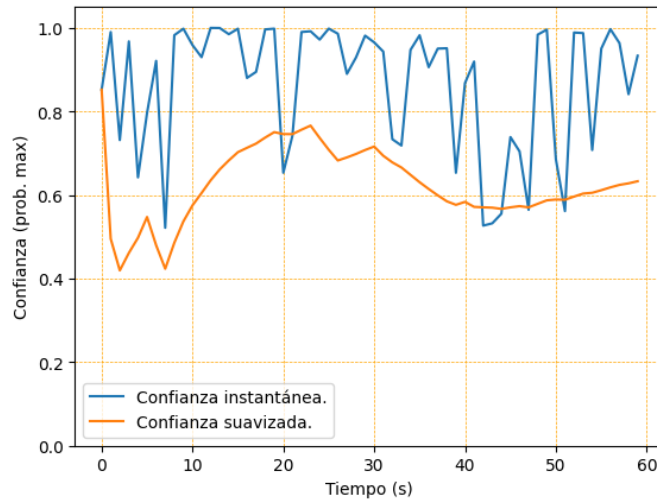
En la figura 3.5 se presenta el comportamiento del modelo en una secuencia continua, evidenciando variabilidad en las predicciones a lo largo del tiempo.

Cuadro 3.1: Arquitectura de la red neuronal convolucional implementada en CNN_1

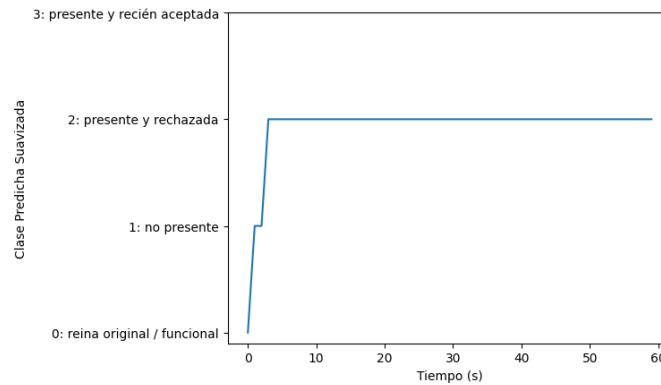
Capa	Tipo	Forma de Salida	Parámetros
Input	–	(3, 32, 201)	–
Conv2D_1	Conv2D + ReLU	(16, 32, 201)	448
BatchNorm_1	Normalización	(16, 32, 201)	64
MaxPool_1	Max Pooling 2×2	(16, 16, 100)	0
Conv2D_2	Conv2D + ReLU	(32, 16, 100)	4,640
BatchNorm_2	Normalización	(32, 16, 100)	128
MaxPool_2	Max Pooling 2×2	(32, 8, 50)	0
Dropout	Dropout	(32, 8, 50)	0
Conv2D_3	Conv2D + ReLU	(64, 8, 50)	18,496
BatchNorm_3	Normalización	(64, 8, 50)	256
MaxPool_3	Max Pooling 2×2	(64, 4, 25)	0
Conv2D_4	Conv2D + ReLU	(128, 4, 25)	73,856
BatchNorm_4	Normalización	(128, 4, 25)	512
GlobalAvgPool	Promedio Global	(128)	0
Dense_1	Fully Connected + ReLU	(64)	8,256
Dropout	Regularización (p=0.35)	(64)	0
Dense_2	Softmax	(4)	260
Total de parámetros:			106,916
*Regularización L2 ($\lambda = 1.5 \times 10^{-4}$) aplicada en Conv2D_1–4 y Dense_1.			

Cuadro 3.2: Métricas de desempeño del modelo CNN_1

Clase	Precision	Recall	F1-score	Soporte
0	0.7847	0.7630	0.7737	1591
1	0.7273	0.7042	0.7156	1413
2	0.6967	0.8295	0.7573	2299
3	0.9383	0.8838	0.9102	6005
Accuracy	0.8333			
Macro Avg	0.7868	0.7951	0.7892	11308
Weighted Avg	0.8412	0.8333	0.8356	11308



(a) Confianza del estado de la reina a lo largo del tiempo.



(b) Estado de la reina suavizado a lo largo del tiempo.

Figura 3.5: Resultados de CNN_1 al clasificar el estado de la reina.

3.2.2. CNN_2: Arquitectura con Activación Sigmoide

La arquitectura CNN_2 introduce modificaciones en la función de activación empleada, reemplazando ReLU por la función sigmoide en las capas convolucionales, el valor relacionado con el Dropout y la ausencia de regularización L2.

Al igual que CNN_1, el modelo está compuesto por bloques convolucionales que incluyen una capa convolucional, seguida de normalización por lotes y operaciones de *max-pooling*. CNN_2 integra una etapa final de *global average pooling*, lo que permite una agregación más robusta de las características espaciales antes de

la clasificación, utilizando el optimizador Adam con una tasa de aprendizaje de 1×10^{-3} .

El desempeño de CNN_2 se evaluó mediante métricas de clasificación, como se muestra en la tabla 3.4, donde alcanza una precisión global (*accuracy*) de 86.58 % (F1-score ponderado de 0.8662), mostrando una mejora tanto en el desempeño global como en la distribución del desempeño entre clases en comparación con CNN_1.

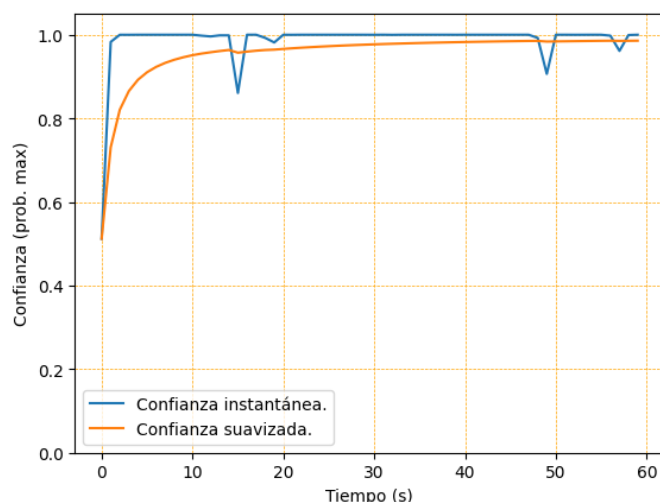
La figura 3.6 muestra la respuesta del modelo en el dominio temporal, evidenciando una mayor estabilidad en las predicciones en comparación con CNN_1.

Cuadro 3.3: Arquitectura de la red neuronal convolucional implementada

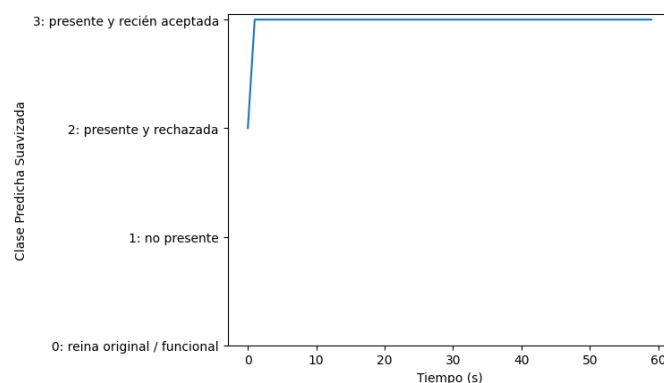
Capa	Tipo	Forma de Salida	Parámetros
Input	–	(3, 32, 201)	–
Conv2D_1	Conv2D + Sigmoid	(16, 32, 201)	448
BatchNorm_1	Normalización	(16, 32, 201)	64
MaxPool_1	Max Pooling 2×2	(16, 16, 100)	0
Conv2D_2	Conv2D + Sigmoid	(32, 16, 100)	4,640
BatchNorm_2	Normalización	(32, 16, 100)	128
MaxPool_2	Max Pooling 2×2	(32, 8, 50)	0
Conv2D_3	Conv2D + Sigmoid	(64, 8, 50)	18,496
BatchNorm_3	Normalización	(64, 8, 50)	256
MaxPool_3	Max Pooling 2×2	(64, 4, 25)	0
Conv2D_4	Conv2D + Sigmoid	(128, 4, 25)	73,856
BatchNorm_4	Normalización	(128, 4, 25)	512
GlobalAvgPool	Promedio Global	(128)	0
Dense_1	Fully Connected + ReLU	(64)	8,256
Dropout	Regularización (p=0.3)	(64)	0
Dense_2	Softmax	(4)	260
Total de parámetros:			106,916

Cuadro 3.4: Métricas de desempeño del modelo CNN_2

Clase	Precision	Recall	F1-score	Soporte
0	0.9175	0.7762	0.8410	1591
1	0.7768	0.8004	0.7884	1413
2	0.7719	0.8317	0.8007	2299
3	0.9144	0.9181	0.9162	6005
Accuracy	0.8658			
Macro Avg	0.8452	0.8316	0.8366	11308
Weighted Avg	0.8687	0.8658	0.8662	11308



(a) Confianza del estado de la reina a lo largo del tiempo.



(b) Estado de la reina suavizado a lo largo del tiempo.

Figura 3.6: Resultados de CNN_2 al clasificar el estado de la reina.

3.3. Distribución del Conjunto de Datos

El conjunto procesado contiene 75,112 segmentos de 2 segundos, distribuidos entre entrenamiento, validación y prueba; tal y como se muestra en la tabla 3.5.

- **Clase 0:** Reina original/funcional (*queen-right*).
- **Clase 1:** Reina no presente (*queenless*).
- **Clase 2:** Reina presente pero rechazada.
- **Clase 3:** Reina presente y recién aceptada.

3.4. Instrumentos

Según Anna Yang (la creadora de la base de datos), para el proceso de recolección de datos, se empleó un dispositivo IoT diseñado a

medida, que incluía un módulo Wi-Fi ESP32, un micrófono digital INMP441, y un sensor de temperatura y humedad BME280. Estos sensores se instalaron dentro de la cubierta telescópi-

Cuadro 3.5: Distribución de segmentos por conjunto y clase

Conjunto	Clase 0	Clase 1	Clase 2	Clase 3	Total
Entrenamiento	7,368	6,535	10,668	27,980	52,551
Validación	1,582	1,400	2,289	5,982	11,253
Prueba	1,591	1,413	2,299	6,005	11,308
Total	10,541	9,348	15,256	39,967	75,112

ca de la colmena (por encima de los cuadros o alzas, pero; por debajo de la cubierta exterior), y se protegieron con una malla metálica para evitar la interferencia de las abejas con los componentes electrónicos, especialmente con el micrófono.

3.5. Base de Datos

El conjunto de datos utilizado en esta investigación fue grabado y publicado por Anna Yang[19]. Contiene datos de audio recolectados de colmenas de abejas melíferas europeas ubicadas en California. Las grabaciones fueron segmentadas en clips de 60 segundos, resultando en un total de 7,100 muestras recolectadas du-

rante un período de cinco semanas, desde el 8 de junio de 2022 (a las 14:52:28 horas) hasta el 15 de julio de 2022 (a las 15:28:21 horas). Cada día contiene aproximadamente entre 23 y 24 muestras.

El proceso de etiquetado se basó en inspecciones manuales diarias de las colmenas, durante las cuales se observó el comportamiento de las abejas hacia la reina (por ejemplo, formación de enjambre o conducta tranquila). En aquellos casos en los que el estado de la reina cambió entre inspecciones, generando ambigüedad sobre el momento exacto de aceptación, las muestras correspondientes fueron descartadas para mantener la integridad del etiquetado.

4. RESULTADOS

4.1. Resultados del Entrenamiento

El entrenamiento para CNN_1 y CNN_2 se realizó tomando un valor máximo de 40 épocas, con *early stopping* (EarlyStopping-*'patience=10'*) y ajuste dinámico de la tasa de aprendizaje.

El mejor desempeño en validación se obtuvo para el modelo CNN_2 en la época 27:

- **Precisión de validación:** 87.71 % .
- **Pérdida de validación:** 0.6462 .

CNN_2 mostró una tendencia de sobreajuste

moderado, evidenciada por la diferencia entre la precisión de entrenamiento (99.21 %) y validación (87.71 %). Sin embargo, la aplicación de regularización mediante *dropout* y pesos de clase permitió mantener un desempeño aceptable.

4.2. Evaluación en el Conjunto de Prueba

CNN_2 destaca entre los dos modelos con una precisión global de **86.58 %** sobre 11,308 segmentos. En este sentido, la matriz de confusión que se muestra en la tabla 4.1 expone que CNN_2 presenta mayor desempeño en la clase 3 y menor en la clase 1.

Cuadro 4.1: Matriz de confusión del modelo en el conjunto de prueba

Clase Real	Clase Predicha				Total
	0	1	2	3	
0 (Queenright)	1,235	95	88	173	1,591
1 (No presente)	42	1,131	147	93	1,413
2 (Rechazada)	50	87	1,912	250	2,299
3 (Aceptada)	19	143	330	5,513	6,005
Total predicho	1,346	1,456	2,477	6,029	11,308

Las métricas de la tabla 4.2 reflejan un comportamiento equilibrado, con variaciones entre precisión y recall según la clase.

Cuadro 4.2: Métricas de clasificación por clase

Clase	Precisión	Recall	F1-Score	Soporte
0 (Queenright)	0.9175	0.7762	0.8410	1,591
1 (No presente)	0.7768	0.8004	0.7884	1,413
2 (Rechazada)	0.7719	0.8317	0.8007	2,299
3 (Aceptada)	0.9144	0.9181	0.9162	6,005
Promedio macro	0.8452	0.8316	0.8366	—
Promedio ponderado	0.8687	0.8658	0.8662	—
Precisión global	0.8658			11,308

4.3. Análisis Temporal de Predicciones

Las figuras 3.5 y 3.6 presentan la evolución temporal de las predicciones sobre grabaciones continuas. CNN_1 (figura 3.5a) muestra mayor variabilidad en la confianza, con valores entre 0.3

y 1.0, mientras que CNN_2 (figura 3.6a) mantiene niveles superiores al 95 % con únicamente dos caídas puntuales, reflejando mayor estabilidad temporal. Esta diferencia se evidencia también en las predicciones suavizadas (figuras 3.5b y 3.6b), donde CNN_2 presenta transiciones entre estados más claras y definidas.

5. DISCUSIÓN

5.1. Interpretación de los Resultados

El uso de arquitecturas basadas en CNN sobre representaciones MFCC demostró ser efectivo para la clasificación del estado de la reina, alcanzando niveles de precisión comparables con trabajos previos, aunque bajo un planteamiento más complejo.

El análisis temporal permitió identificar diferencias relevantes que no son capturadas por métricas globales.

5.1.1. Comportamiento de CNN_1

Al observar la figura 3.5a, se evidenció que CNN_1 presenta alta variabilidad en la confianza, lo que sugiere sensibilidad a fluctuaciones locales del audio. Las predicciones suavizadas según la figura 3.5b, muestran transiciones frecuentes, lo que dificulta la identificación de períodos prolongados en donde se pueda determinar con precisión la presencia de una reina, limitando así la utilidad práctica de CNN_1 para un monitoreo en tiempo real.

5.1.2. Comportamiento de CNN_2

Con base en la figura 3.6a, se observó que CNN_2 presenta mayor estabilidad temporal, con confianza sostenida y transiciones más claras. Las predicciones suavizadas que se muestran en la figura 3.6b presentan un comportamiento estable al momento de identificar la clase. Sin embargo, se observan caídas puntuales que indican eventos de incertidumbre o limitaciones del modelo.

5.2. Análisis Comparativo

Dado que ambos modelos fueron desarrollados bajo un mismo esquema base, las diferencias observadas pueden atribuirse principalmente a la profundidad de la red y al mecanismo de reducción espacial.

CNN_2 logra una representación más robusta, reflejada tanto en métricas globales como en estabilidad temporal. No obstante, los errores observados indican que el problema aún presenta desafíos asociados a la variabilidad acústica.

5.3. Comparación con el Estado del Arte

Al analizar los resultados de ambos modelos se observa que CNN_2 ha logrado superar las limitaciones de CNN_1, alcanzando una mayor estabilidad y precisión en la clasificación del estado de la reina. Esta mejora no solo se debe al cambio en la función de activación, sino también a modificaciones en la arquitectura, incluyendo una mayor profundidad y la incorporación de una capa de *global average pooling*. En conjunto, estas decisiones permiten al modelo aprender representaciones más robustas y generalizables. Sin embargo, la presencia de eventos de incertidumbre en ambos modelos indica que aún existen desafíos relacionados con la variabilidad acústica y las condiciones de grabación, lo que resalta la necesidad de continuar explorando técnicas para mejorar la robustez y generalización de un modelo para la identificación del estado de la reina.

En conjunto, los resultados indican que CNN_2 presenta un mejor desempeño global y una distribución más equilibrada entre clases en comparación con CNN_1. La mayor profundidad de la red y el uso de *global average pooling* contribuyen a una mejor representación de las

características. No obstante, el análisis temporal revela que esta mejora no es completamente consistente en todos los escenarios, ya que el modelo aún presenta errores en ciertas secuencias, lo que evidencia limitaciones en su capacidad de generalización. Esto sugiere que, aunque CNN_2 es una mejora respecto a CNN_1, aún existen oportunidades para optimizar la arquitectura y el proceso de entrenamiento con el fin de lograr un desempeño más robusto y consistente en la clasificación del estado de la reina a lo largo del tiempo.

5.3.1. Comparación con el Estado del Arte

Los resultados que se obtuvieron a partir de CNN_2, aunque inferiores en términos de precisión global a los reportados por Ruvunga et al. [4, 9], resultan relevantes considerando las diferencias en el planteamiento del problema y en la representación de entrada. Una de las diferencias más significativas es el uso de MFCCs en lugar de espectrogramas STFT, lo que proporciona una representación más compacta y perceptualmente relevante del audio. Además, este trabajo aborda un problema de clasificación multiclase con cuatro categorías, en contraste con los enfoques binarios comúnmente utilizados en la literatura.

- **Representación de entrada:** a diferencia de los espectrogramas STFT utilizados en trabajos previos, el uso de MFCCs proporciona una representación más compacta y perceptualmente relevante.
- **Clasificación multiclase:** este trabajo aborda un problema más complejo con cuatro clases, en contraste con enfoques binarios.
- **Análisis temporal:** se incorpora evaluación de consistencia temporal, aspecto poco explorado en estudios previos.

5.4. Limitaciones y Desafíos Identificados

5.4.1. Generalización

Los datos provienen de condiciones controladas, por lo que la generalización a otros entornos requiere validación adicional. Factores como clima, raza de abejas y condiciones ambientales pueden influir en los patrones acústicos [11].

5.4.2. Desbalance de Clases

El dataset presenta desbalance al mostrar mayor cantidad de datos para una clase con respecto a las demás, lo cual puede afectar el desempeño. La cantidad de etiquetas por clase se muestra a continuación:

- queenright/original queen (0): 179.
- not present (1): 158.
- present and rejected (2): 259.
- present and newly accepted (3): 679.

5.4.3. Robustez al Ruido

El ruido ambiental puede degradar el desempeño del modelo [14]. Técnicas de denoising o modelos más robustos podrían ser necesarios para aplicaciones en campo.

5.4.4. Implicaciones Prácticas

Los resultados demuestran la viabilidad de sistemas de monitoreo acústico no invasivo.

- Reducción de inspecciones invasivas.
- Detección temprana de la ausencia de una reina.
- Escalabilidad mediante dispositivos IoT.
- Contribución a la sustentabilidad apícola [10, 11].

5.4.5. Direcciones Futuras

Se proponen las siguientes líneas de investigación:

- Validación con múltiples datasets.
- Modelos híbridos CNN-LSTM.
- Transfer learning.
- Detección de eventos acústicos específicos.
- Implementación en hardware embebido.
- Integración multimodal.

6. CONCLUSIONES

La incorporación de un análisis temporal de la confianza y de las predicciones suavizadas constituye una de las principales contribuciones de este trabajo, ya que permite evaluar la consistencia de las decisiones del modelo a lo largo del tiempo, un aspecto escasamente abordado en estudios previos. Este enfoque complementa las métricas tradicionales y acerca la evaluación del modelo a escenarios reales de monitoreo continuo.

En relación con la hipótesis planteada, los resultados obtenidos confirman que el comportamiento acústico de una colmena varía en función del estado de la abeja reina, y que dichas variaciones pueden ser capturadas mediante características espectrales como los coeficientes cepstrales en escala Mel (MFCC). Esto valida el uso del análisis acústico como una herramienta viable para la detección automática del estado de la reina.

La abeja reina es un elemento fundamental para la estabilidad y supervivencia de la colonia, ya que su presencia regula el comportamiento colectivo de las abejas. Su ausencia o fallo puede derivar en el debilitamiento progresivo o incluso en el colapso de la colmena. En este contexto, el desarrollo de herramientas no invasivas para la detección temprana de su estado representa un avance significativo para la apicultura moderna.

En este trabajo se demostró que es posible abordar la clasificación del estado de la reina

mediante técnicas de aprendizaje profundo, alcanzando una precisión global del 86.58 % sobre un conjunto de prueba independiente. Este resultado respalda la viabilidad del enfoque propuesto y su potencial aplicación en sistemas reales.

Asimismo, el diseño experimental permitió analizar el impacto de distintas decisiones arquitectónicas, incluyendo la función de activación, la profundidad de la red y el uso de *global average pooling*. Los resultados evidencian que estas decisiones no solo afectan la capacidad de discriminación entre clases, sino también la estabilidad temporal de las predicciones, un factor crítico en aplicaciones de monitoreo continuo.

En conjunto, los hallazgos de este estudio amplían el entendimiento sobre el uso de modelos de aprendizaje profundo en bioacústica, particularmente en el monitoreo de colmenas. No obstante, también se identifican áreas de mejora, como la robustez ante ruido, la generalización a diferentes condiciones ambientales y la incorporación de modelos que capturen dependencias temporales más complejas.

Finalmente, este trabajo sienta las bases para el desarrollo de sistemas de monitoreo acústico no invasivo que permitan asistir a los apicultores en la toma de decisiones, reduciendo la necesidad de inspecciones físicas y contribuyendo a la sostenibilidad y conservación de las colonias de abejas.

Bibliografía

- [1] Sofía de León Guedes, Estela Santos, Arnaldo Moreni et al. «Calidad nutricional del polen de colza (*Brassica napus*) y determinación de la flora apícola acompañante para las abejas melíferas». En: (2023).
- [2] ML Winston. *Temperate and Tropical Honey Bees in The Biology of the Honey Bee*. 214–224. 1987.
- [3] M. Beekman y F. L. W. Ratnieks. «Long-range foraging by the honey-bee, *Apis mellifera* L». En: *Functional Ecology* 14 (2000), págs. 490-496. DOI: [10.1046/j.1365-2435.2000.00443.x](https://doi.org/10.1046/j.1365-2435.2000.00443.x).
- [4] Stenford Ruvinga et al. «Use of LSTM networks to identify “queenlessness” in honeybee hives from audio signals». En: *2021 17th International Conference on Intelligent Environments (IE)*. IEEE. 2021, págs. 1-4.
- [5] Zhengwei Wang et al. «Encoding and decoding of the information in the honeybee waggle dance». En: *Behavioral Ecology and Sociobiology* 79.5 (2025), pág. 53.
- [6] Brian R Johnson. «Division of labor in honeybees: form, function, and proximate mechanisms». En: *Behavioral ecology and sociobiology* 64 (2010), págs. 305-316.
- [7] Hichem Maaref y Dominique Fourer. «Remote Beehive Health Analysis using Embedded System and Relevant Audio Features». En: ().
- [8] Donald Howard et al. «Signal processing the acoustics of honeybees (*apis mellifera*) to identify the ‘queenless’ state in hives». En: *Proceedings of the Institute of Acoustics* 35.1 (2013), pág. 290.
- [9] Stenford Ruvinga et al. «Identifying queenlessness in honeybee hives from audio signals using machine learning». En: *Electronics* 12.7 (2023), pág. 1627.
- [10] Shaden AM Khalifa et al. «Overview of bee pollination and its economic value for crop production». En: *Insects* 12.8 (2021), pág. 688.
- [11] Simon G Potts et al. «Safeguarding pollinators and their values to human well-being». En: *Nature* 540.7632 (2016), págs. 220-229.
- [12] Sara Ferrari et al. «Monitoring of swarming sounds in bee hives for early detection of the swarming period». En: *Computers and electronics in agriculture* 64.1 (2008), págs. 72-77.

- [13] Antonio Robles Guerrero et al. «Análisis de patrones acústicos para determinar estado de colonias de abejas mediante Regresión Logística Lasso y Descomposición en Valores Singulares». En: (2019).
- [14] Rui Peng, Iman Ardekani y Hamid Sharifzadeh. «An acoustic signal processing system for identification of queen-less beehives». En: *2020 Asia-Pacific Signal and Information Processing Association Annual Summit and Conference (APSIPA ASC)*. IEEE. 2020, págs. 57-63.
- [15] Stanley Smith Stevens, John Volkman y Edwin Broomell Newman. «A scale for the measurement of the psychological magnitude pitch». En: *The journal of the acoustical society of america* 8.3 (1937), págs. 185-190.
- [16] H Shimodaira y S Rennals. *Speech Signal Analysis*. <https://www.inf.ed.ac.uk/teaching/courses/asr/2012-13/asr02-signal-4up.pdf>. 2013.
- [17] Fang Zheng, Guoliang Zhang y Zhanjiang Song. «Comparison of different implementations of MFCC». En: *Journal of Computer Science and Technology* 16.6 (2001), págs. 582-589.
- [18] Sepp Hochreiter y Jürgen Schmidhuber. «Long short-term memory». En: *Neural computation* 9.8 (1997), págs. 1735-1780.
- [19] Anna Yang. *Smart Bee Colony Monitor: Clips of Beehive Sounds*. <https://www.kaggle.com/datasets/annajyang/beehive-sounds>. [Online; accessed 29-April-2025]. 2023.

APÉNDICE A

NOTA: los acentos fueron removidos en la descripción de los programas por problemas de compatibilidad con la librería 'listing' en L^AT_EX.

Programa # 1

```
1 import numpy as np
2 import librosa
3 import librosa.display
4 import matplotlib.pyplot as plt
5 from pathlib import Path
6
7 def peak_normalize(x, eps=1e-9):
8     '''
9     Normalizacion por pico (peak normalization).
10    Se aplica despues de trim y antes de segmentar (a nivel de archivo completo)
11    .
12    - Args:
13        x: senal de audio (1D numpy array)
14        eps: epsilon para evitar division por cero
15    - Returns:
16        Audio normalizado con pico = 1.0
17    '''
18    return x / (np.max(np.abs(x)) + eps)
19
20 def segment_signal(x, sr, seg_sec, hop_sec):
21     '''
22     Segmenta la senal de audio en ventanas con overlap.
23     Si un segmento es mas corto que seg_sec, se hace padding con reflect.
24    - Args:
25        x: senal de audio (1D numpy array)
26        sr: sample rate
27        seg_sec: duracion del segmento en segundos
28        hop_sec: hop entre segmentos en segundos
29    - Returns:
30        Lista de segmentos de audio
31    '''
32    seg_len = int(seg_sec * sr)
33    hop_len = int(hop_sec * sr)
```

```
33
34 # Si el audio es mas corto que seg_len, hacer padding
35 if len(x) < seg_len:
36     x = np.pad(x, (0, seg_len - len(x)), mode='reflect')
37
38 segments = []
39 for start in range(0, max(1, len(x) - seg_len + 1), hop_len):
40     seg = x[start:start + seg_len]
41     if len(seg) < seg_len:
42         seg = np.pad(seg, (0, seg_len - len(seg)), mode='reflect')
43     segments.append(seg)
44 return segments
45
46 def extract_mfcc_features(audio_segment):
47     '''
48     Extrae MFCCs siguiendo el pipeline de produccion:
49     1. Extrae N_MFCC_RAW (33) MFCCs
50     2. Elimina C0 (primer coeficiente) - queda (32, T)
51     3. Calcula delta y delta-delta sobre los 32 coeficientes
52     4. Apila - (3, 32, T) si ADD_DELTAS=True
53     - Args:
54         audio_segment: senal de audio (1D numpy array)
55     - Returns:
56         features: numpy array con shape (channels, N_MFCC_FINAL, T_frames)
57                 channels = 3 si ADD_DELTAS=True, else 1
58     '''
59     # Extraer MFCCs con C0 incluido
60     mfcc = librosa.feature.mfcc(
61         y=audio_segment,
62         sr=SR,
63         n_mfcc=N_MFCC_RAW,      # 33 coeficientes
64         n_fft=N_FFT,
65         hop_length=HOP_LEN,
66         fmin=FMIN,
67         fmax=FMAX
68     )
69     # Eliminar C0 (primer coeficiente) - shape: (32, T)
70     mfcc = mfcc[1:, :]
71
72     if ADD_DELTAS:
73         # Calcular deltas sobre los 32 coeficientes (sin C0)
74         delta = librosa.feature.delta(mfcc)
75         delta2 = librosa.feature.delta(mfcc, order=2)
```

```
76         # Apilar: (3, 32, T)
77         features = np.stack([mfcc, delta, delta2], axis=0)
78     else:
79         # Solo MFCCs: (1, 32, T)
80         features = mfcc[np.newaxis, :, :]
81     return features.astype(np.float32)
82
83 def preprocess_audio_file(audio_path):
84     '''
85     Pipeline completo de procesamiento consistente con produccion:
86     - Args:
87         audio_path: ruta al archivo de audio
88     - Returns:
89         segments_features: lista de arrays con shape (3, 32, T_frames)
90         metadata: diccionario con informacion del procesamiento
91     '''
92     audio_path = Path(audio_path)
93
94     # 1. Load + resample
95     audio_raw, _ = librosa.load(str(audio_path), sr=SR, mono=True)
96
97     # 2. Trim
98     audio_trimmed, _ = librosa.effects.trim(audio_raw, top_db=TRIM_DB)
99
100    # 3. Peak normalization (a nivel de archivo)
101    audio_normalized = peak_normalize(audio_trimmed)
102
103    # 4. Segmentar
104    segments = segment_signal(audio_normalized, SR, SEG_SEC, HOP_SEC)
105
106    # 5. Extraer features por segmento
107    segments_features = []
108    for i, seg in enumerate(segments):
109        feat = extract_mfcc_features(seg)
110        segments_features.append(feat)
111
112    # Metadata
113    metadata = {
114        'filename': audio_path.name,
115        'duration_original': len(audio_raw) / SR,
116        'duration_trimmed': len(audio_trimmed) / SR,
117        'num_segments': len(segments),
118        'feature_shape': segments_features[0].shape if segments_features else
```



```

    None
119     }
120     return segments_features, metadata
121
122 # PARAMETROS DE PREPROCESAMIENTO
123
124 # Audio Configuration
125 SR = 16000          # Sample rate (resample todo a 16kHz)
126 TRIM_DB = 30        # Top dB para trim (eliminar silencios)
127
128 # Segmentacion
129 SEG_SEC = 2.0        # Duracion de cada segmento: 2 segundos
130 HOP_SEC = 1.0        # Hop entre segmentos: 1 segundo (50% overlap)
131
132 # MFCC FE
133 # Ventana: 25ms, Hop: 10ms (estandar en ASR)
134 N_FFT = int(0.025 * SR)    # 25ms - 400 muestras
135 HOP_LEN = int(0.010 * SR)  # 10ms - 160 muestras
136
137 # Frecuencias
138 FMIN = 20            # Frecuencia minima (Hz)
139 FMAX = SR // 2       # Frecuencia maxima (Nyquist): 8000 Hz
140
141 # Coeficientes MFCC
142 N_MFCC_RAW = 33       # Extraemos 33 MFCCs inicialmente
143 N_MFCC_FINAL = 32     # Eliminamos C0 - 32 coeficientes finales
144 ADD_DELTAS = True     # Incluir delta y delta-delta
145
146 AUDIO_PATH = r'<<archivo de audio>>'
147 features, metadata = preprocess_audio_file(AUDIO_PATH, verbose=True)
148
149 # Directorio con audios
150 DATA_DIR = Path(r'<<directorio con archivos de audio>>')
151
152 # Buscar los primeros 5 archivos WAV
153 audio_files = list(DATA_DIR.rglob('*.wav'))[:5]
154
155 if len(audio_files) == 0:
156     print('No se encontraron archivos .wav en el directorio.')
157 else:
158     print(f'\n{'='*70}')
159     print(f'TEST BATCH: Procesando {len(audio_files)} archivos')
160     print(f'{'='*70}\n')
```

```
161
162     results = []
163     for audio_path in audio_files:
164         try:
165             features, metadata = preprocess_audio_file(audio_path, verbose=False
166 )
167             results.append({
168                 'filename': metadata['filename'],
169                 'num_segments': metadata['num_segments'],
170                 'shape': metadata['feature_shape']
171             })
172             print(f'{metadata["filename"]:<30} - {metadata["num_segments"]:>3}
173 segmentos')
174         except Exception as e:
175             print(f'{audio_path.name:<30} - ERROR: {e}')
176
177     print(f'\n{'='*70}')
178     print(f'Procesamiento batch completado: {len(results)}/{len(audio_files)}
179 exitosos')
180     print(f'\n{'='*70}')
181
182 # Cargar un segmento del dataset de entrenamiento
183 NPY_DIR = Path(r'<<directorio con archivo *.np>>')
184
185 try:
186     X_train = np.load(NPY_DIR / 'X_train.npy', mmap_mode='r')
187     print(f'\n{'='*70}')
188     print('VALIDACION DE CONSISTENCIA')
189     print(f'\n{'='*70}')
190     print(f'Shape del dataset de entrenamiento: {X_train.shape}')
191     print(f'    Esperado: (N_samples, 3, 32, T_frames)')
192     print(f'\nShape de nuestro test:')
193     if 'features' in locals() and len(features) > 0:
194         print(f'    {features[0].shape}')
195         print(f'\n Las dimensiones coinciden.' if features[0].shape[0] == 3 and
196 features[0].shape[1] == 32 else 'Las dimensiones NO coinciden')
197
198 # Estadísticas comparativas
199 print(f'\n Estadísticas de X_train (primer segmento):')
200 sample = X_train[0].astype(np.float32)
201 print(f'    Min:  {np.min(sample):.4f}')
202 print(f'    Max:  {np.max(sample):.4f}')
203 print(f'    Mean: {np.mean(sample):.4f}')
```

```
200     print(f'    Std:  {np.std(sample):.4f}')
201     if 'features' in locals() and len(features) > 0:
202         print(f'\n Estadísticas de nuestro test (primer segmento):')
203         print(f'    Min:  {np.min(features[0]):.4f}')
204         print(f'    Max:  {np.max(features[0]):.4f}')
205         print(f'    Mean: {np.mean(features[0]):.4f}')
206         print(f'    Std:  {np.std(features[0]):.4f}')
207     print(f'{'='*70}\n')
208
209 except FileNotFoundError:
210     print('No se encontraron los archivos .npy de entrenamiento.')
```

Listing A.1: Prueba de extracción de características acústicas (MFCC) y validación de la representación numérica del audio.o

Programa # 2

```
1 # CELDA 1:
2 # -----
3 from pathlib import Path
4 import pandas as pd
5 import numpy as np
6 import librosa
7 from sklearn.model_selection import train_test_split
8 import json
9 # -----
10
11 # CELDA 2:
12 # -----
13 def peak_normalize(x, eps=1e-9):
14     return x / (np.max(np.abs(x)) + eps)
15
16 def load_and_clean(path):
17     '''
18     Preprocesamiento a nivel de archivo (CONTRATO DE PREPROCESAMIENTO):
19     1. Load + resample a SR=16000
20     2. Trim con TRIM_DB=30 (elimina silencios al inicio/fin)
21     3. Peak normalization sobre el audio TRIMMEADO (a nivel de archivo completo)
22     '''
23     x, _ = librosa.load(str(path), sr=SR, mono=True)
24     x, _ = librosa.effects.trim(x, top_db=TRIM_DB)
25     x = peak_normalize(x)
26     return x
27
28 def segment_signal(x, sr, seg_sec, hop_sec):
29     seg_len = int(seg_sec * sr)
30     hop_len = int(hop_sec * sr)
31     if len(x) < seg_len:
32         x = np.pad(x, (0, seg_len - len(x)), mode='reflect')
33     segments = []
34     for start in range(0, max(1, len(x)-seg_len+1), hop_len):
35         seg = x[start:start+seg_len]
36         if len(seg) < seg_len:
37             seg = np.pad(seg, (0, seg_len - len(seg)), mode='reflect')
38         segments.append(seg)
39     return segments
40
41 def mfcc_features(seg):
42     '''
```

```

43     Extrae MFCCs + deltas con C0 eliminado.
44     Contrato de preprocesamiento:
45     1. Se recibe 'seg' ya trimmeado, normalizado y segmentado
46     2. Extraemos 33 MFCCs (n_mfcc=33)
47     3. Eliminamos C0 -> queda (32, T)
48     4. Calculamos Delta y Delta-Delta sobre los 32 coefs restantes
49     5. Apilamos -> (3, 32, T)
50     '''
51     # Extraer 33 MFCCs para luego eliminar C0
52     mfcc = librosa.feature.mfcc(
53         y=seg, sr=SR, n_mfcc=33, n_fft=N_FFT, hop_length=HOP_LEN,
54         fmin=FMIN, fmax=FMAX
55     )
56     # Eliminar C0 (primer coeficiente) -> ahora (32, T)
57     mfcc = mfcc[1:, :]
58
59     if ADD_DELTAS:
60         d1 = librosa.feature.delta(mfcc)
61         d2 = librosa.feature.delta(mfcc, order=2)
62         feat = np.stack([mfcc, d1, d2], axis=0) # (3, 32, T)
63     else:
64         feat = mfcc[np.newaxis, :, :] # (1, 32, T)
65     return feat.astype(np.float32)
66 # -----
67
68 # CELDA 3:
69 # -----
70 DATA_DIR = Path(r'<<directorio con archivos de audio>>')
71 CSV_PATH = Path(r'<<archivo *.csv con etiquetas>>')
72
73 ID_COL = 'file name' # <-- columna de nombres
74 TARGET_COL = 'queen status' # <-- la etiqueta que se utilizara
75 VALID_CLASSES = {0, 1, 2, 3}
76
77 # 1) Construir diccionario: nombre_archivo.wav -> ruta_completa
78 wav_paths = sorted(list(DATA_DIR.rglob('*.wav')) + sorted(list(DATA_DIR.rglob('
79     *.WAV'))))
80 name_to_path = {p.name: str(p) for p in wav_paths} # str() para que sea
81     serializable y facil
82
83 df = pd.read_csv(CSV_PATH)
84
85 assert ID_COL in df.columns, f'Falta ID_COL={ID_COL} en el CSV'

```

```

84 assert TARGET_COL in df.columns, f'Falta TARGET_COL={TARGET_COL} en el CSV'
85
86 # 2) Normaliza nombres (solo basename)
87 df['_basename'] = df[ID_COL].astype(str).map(lambda s: Path(s).name)
88
89 # 3) Convertir queen status a numerico y filtrar clases validas
90 df['_y'] = pd.to_numeric(df[TARGET_COL], errors='coerce')
91 labeled = df[df['_y'].isin([0,1,2,3])].copy()
92 labeled['y'] = labeled['_y'].astype(np.int64)
93
94 # 4) Crear columna 'path' (ruta completa al wav)
95 labeled['path'] = labeled['_basename'].map(name_to_path)
96
97 # 5) Verificar que no falte ninguno
98 missing = labeled['path'].isna().sum()
99 print('Etiquetados:', len(labeled), '| faltantes sin path:', int(missing))
100 assert missing == 0, 'Hay archivos etiquetados que no se encontraron en la
    carpeta.'
101 print('Conteo por clase:\n', labeled['y'].value_counts().sort_index())
102
103 # Audio / MFCC params
104 SR = 16000
105 TRIM_DB = 30
106 SEG_SEC = 2.0
107 HOP_SEC = 1.0
108 N_MFCC = 32
109 N_FFT = int(0.025 * SR)
110 HOP_LEN= int(0.010 * SR)
111 FMIN, FMAX = 20, SR//2
112 ADD_DELTAS = True
113 RANDOM_SEED = 123
114 TEST_SIZE = 0.15
115 VAL_SIZE = 0.15
116
117 # Direccion para guardar los archivos .npy con las caracteristicas MFCC
118 OUT_DIR = Path(r'<<directorio con archivos de audio>>')
119 OUT_DIR.mkdir(parents=True, exist_ok=True)
120 # -----
121
122 # CELDA 4:
123 # -----
124 paths = labeled['path'].values
125 ys = labeled['y'].values

```

```
126
127 # paths, ys ya definidos (por archivo, no por segmento)
128 paths_train, paths_tmp, y_train, y_tmp = train_test_split(
129     paths, ys, test_size=(TEST_SIZE + VAL_SIZE), random_state=RANDOM_SEED,
130     stratify=ys
131 )
132 # aqui tmp es (val+test); lo partimos a la mitad si VAL_SIZE==TEST_SIZE
133 rel_test = TEST_SIZE / (TEST_SIZE + VAL_SIZE)
134
135 paths_val, paths_test, y_val, y_test = train_test_split(
136     paths_tmp, y_tmp, test_size=rel_test, random_state=RANDOM_SEED, stratify=
137     y_tmp
138 )
139 print('Train/Val/Test:', len(paths_train), '/', len(paths_val), '/', len(
140     paths_test))
141 # -----
142 # CELDA 5:
143 # -----
144 def process_labeled_file_list(file_list, label_list):
145     feats = []
146     labs = []
147     file_index = [] # trazabilidad: (archivo, segmento)
148     for p, y in zip(file_list, label_list):
149         x = load_and_clean(p)
150         segs = segment_signal(x, SR, SEG_SEC, HOP_SEC)
151         for k, seg in enumerate(segs):
152             feats.append(mfcc_features(seg))
153             labs.append(y)
154             file_index.append({'file': str(p), 'segment': int(k)})
155     X = np.stack(feats, axis=0) # (Nseg, C, n_mfcc, T)
156     y = np.array(labs, dtype=np.int64)
157     return X, y, file_index
158
159 print('Extrayendo TRAIN...')
160 X_train, y_train_seg, idx_train = process_labeled_file_list(paths_train, y_train
161 )
162 print('Extrayendo VAL...')
163 X_val, y_val_seg, idx_val = process_labeled_file_list(paths_val, y_val)
164 print('Extrayendo TEST...')
165 X_test, y_test_seg, idx_test = process_labeled_file_list(paths_test, y_test)
```

```
165
166 print('Shapes:')
167 print('X_train:', X_train.shape, 'y:', y_train_seg.shape)
168 print('X_val:', X_val.shape, 'y:', y_val_seg.shape)
169 print('X_test:', X_test.shape, 'y:', y_test_seg.shape)
170 # -----
171
172 # CELDA 6:
173 # -----
174 X_train = X_train.astype(np.float16)
175 X_val    = X_val.astype(np.float16)
176 X_test   = X_test.astype(np.float16)
177
178 np.save(OUT_DIR / 'X_train.npy', X_train)
179 np.save(OUT_DIR / 'y_train.npy', y_train_seg)
180 np.save(OUT_DIR / 'X_val.npy', X_val)
181 np.save(OUT_DIR / 'y_val.npy', y_val_seg)
182 np.save(OUT_DIR / 'X_test.npy', X_test)
183 np.save(OUT_DIR / 'y_test.npy', y_test_seg)
184 # -----
```

Listing A.2: Generando archivos *.npy (En Jupyter Notebooks *.ipynb).

Programa # 3

```
1 # CELDA 1:
2 # -----
3 from pathlib import Path
4 import os, random
5 import numpy as np
6 import tensorflow as tf
7 from tensorflow.keras import layers, models
8 from sklearn.metrics import classification_report, confusion_matrix
9
10 SEED = 123
11
12 os.environ['PYTHONHASHSEED'] = str(SEED)
13 random.seed(SEED)
14 np.random.seed(SEED)
15 tf.random.set_seed(SEED)
16
17 BASE = Path(r'<<Directorio con archivos *.numpy>>')
18
19 X_train = np.load(BASE / 'X_train.npy', mmap_mode='r')
20 y_train = np.load(BASE / 'y_train.npy')
21 X_val = np.load(BASE / 'X_val.npy', mmap_mode='r')
22 y_val = np.load(BASE / 'y_val.npy')
23 X_test = np.load(BASE / 'X_test.npy', mmap_mode='r')
24 y_test = np.load(BASE / 'y_test.npy')
25
26 print('X_train:', X_train.shape, X_train.dtype)
27 print('y_train:', y_train.shape, y_train.dtype)
28 print('X_val: ', X_val.shape, X_val.dtype)
29 print('X_test: ', X_test.shape, X_test.dtype)
30
31 # Chequeo de clases
32 u, c = np.unique(y_train, return_counts=True)
33 print('Distribucion train:', dict(zip(u, c)))
34 # -----
35
36 # CELDA 2:
37 # -----
38 BATCH = 64
39
40 def make_ds(X, y, training=False):
41     ds = tf.data.Dataset.from_tensor_slices((X, y))
42     if training:
```

```
43     ds = ds.shuffle(20000, reshuffle_each_iteration=True)
44     ds = ds.batch(BATCH)
45     ds = ds.map(lambda a,b: (tf.cast(a, tf.float32), b), num_parallel_calls=tf.
46     data.AUTOTUNE)
47     ds = ds.prefetch(tf.data.AUTOTUNE)
48     return ds
49
50 train_ds = make_ds(X_train, y_train, training=True)
51 val_ds    = make_ds(X_val, y_val, training=False)
52 test_ds   = make_ds(X_test, y_test, training=False)
53
54 # -----
55 # CELDA 3:
56 # -----
57 num_classes = 4
58 counts = np.bincount(y_train, minlength=num_classes)
59 total = counts.sum()
60
61 # Peso inverso a frecuencia (simple y efectivo)
62 class_weight = {i: float(total / (num_classes * counts[i])) for i in range(
63     num_classes) if counts[i] > 0}
64 print('counts:', counts)
65 print('class_weight:', class_weight)
66 # -----
67
68 # CELDA 4:
69 # -----
70 num_classes = 4
71 input_shape = (3, 32, 201)
72 act_fn = 'sigmoid'
73
74 model = models.Sequential([
75     layers.Input(shape=input_shape),
76     layers.Conv2D(16, (3,3), padding='same', activation=act_fn, data_format='
77     channels_first'),
78     layers.BatchNormalization(axis=1),
79     layers.MaxPool2D((2,2), data_format='channels_first'),
80
81     layers.Conv2D(32, (3,3), padding='same', activation=act_fn, data_format='
82     channels_first'),
83     layers.BatchNormalization(axis=1),
84     layers.MaxPool2D((2,2), data_format='channels_first'),
```

```

82     layers.Conv2D(64, (3,3), padding='same', activation=act_fn, data_format='
channels_first'),
83     layers.BatchNormalization(axis=1),
84     layers.MaxPool2D((2,2), data_format='channels_first'),
85
86     layers.Conv2D(128, (3,3), padding='same', activation=act_fn, data_format='
channels_first'),
87     layers.BatchNormalization(axis=1),
88     layers.GlobalAveragePooling2D(data_format='channels_first'),
89
90     layers.Dense(64, activation='relu'),
91     layers.Dropout(0.3),
92     layers.Dense(num_classes, activation='softmax')
93 ])
94
95 model.compile(
96     optimizer=tf.keras.optimizers.Adam(1e-3),
97     loss='sparse_categorical_crossentropy',
98     metrics=['accuracy']
99 )
100
101 model.summary()
102 # -----
103
104 # CELDA 5:
105 # -----
106 callbacks = [
107     tf.keras.callbacks.EarlyStopping(
108         monitor='val_accuracy', mode='max',
109         patience=10, restore_best_weights=True
110     ),
111     tf.keras.callbacks.ReduceLROnPlateau(
112         monitor='val_accuracy', mode='max',
113         patience=3, factor=0.5
114     ),
115     tf.keras.callbacks.ModelCheckpoint(
116         filepath=str(BASE / 'CNN\_mfcc\_queenRec1.h5'),
117         monitor='val_accuracy', mode='max',
118         save_best_only=True
119     )
120 ]
121
122 history = model.fit(

```

```
123     train_ds ,
124     validation_data=val_ds ,
125     epochs=40 ,
126     class_weight=class_weight ,
127     callbacks=callbacks
128 )
129 # -----
130
131 # CELDA 6:
132 # -----
133 probs = model.predict(test_ds)
134 y_pred = np.argmax(probs, axis=1)
135
136 print('Confusion matrix:\n', confusion_matrix(y_test, y_pred))
137 print('\nClassification report:\n', classification_report(y_test, y_pred, digits
    =4))
138 # -----
```

Listing A.3: Extracción de características.

Programa # 3

```
1 # CELDA 1:
2 # -----
3 def peak_normalize(x, eps=1e-9):
4     m = np.max(np.abs(x))
5     return x if m < eps else (x / (m + eps))
6
7 def extract_mfcc_3ch(window_audio: np.ndarray) -> np.ndarray:
8     '''
9     Input: 1D float array of length WINDOW_SEC*SR
10    Output: (3, 32, 201) float32 [channels_first]
11    '''
12    # Ajustar normalizacion de amplitud al estilo de entrenamiento (aprox.)
13    w = peak_normalize(window_audio.astype(np.float32))
14
15    mfcc = librosa.feature.mfcc(
16        y=w, sr=SR,
17        n_mfcc=N_MFCC,
18        n_fft=N_FFT,
19        hop_length=HOP_LEN,
20        fmin=FMIN, fmax=FMAX
21    ) # (32, T)
22
23    if ADD_DELTAS:
24        d1 = librosa.feature.delta(mfcc)
25        d2 = librosa.feature.delta(mfcc, order=2)
26        feat = np.stack([mfcc, d1, d2], axis=0) # (3, 32, T)
27    else:
28        feat = mfcc[np.newaxis, :, :] # (1, 32, T)
29
30    return feat.astype(np.float32)
31 # -----
32
33 # CELDA 2:
34 # -----
35 import time
36 import numpy as np
37 import librosa
38 import matplotlib.pyplot as plt
39 from pathlib import Path
40 from collections import deque
41 import tensorflow as tf
42
```

```
43 # -----
44 # Direcciones
45 # -----
46 MODEL_PATH = Path(r'<<Modelo (red neuronal)>>')
47 WAV_PATH    = Path(r'<<Archivo *.wav')
48
49 assert MODEL_PATH.exists(), f'Model not found: {MODEL_PATH}'
50 assert WAV_PATH.exists(),   f'WAV not found: {WAV_PATH}'
51
52 # -----
53 # Parametros
54 # -----
55 SR = 16000
56 WINDOW_SEC = 2.0
57 HOP_SEC = 1.0
58 SMOOTH_SEC = 60.0
59
60 # Parametros MFCC
61 N_MFCC = 32
62 N_FFT = int(0.025 * SR)      # 25 ms -> 400
63 HOP_LEN = int(0.010 * SR)    # 10 ms -> 160
64 FMIN, FMAX = 20, SR // 2
65 ADD_DELTAS = True
66
67 SIMULATE_REALTIME = True     # True = sleep 1s per hop, False = run fast
68
69 # Mapeo de etiquetas
70 CLASS_NAME = {
71     0: '0: reina original / funcional',
72     1: '1: no presente',
73     2: '2: presente y rechazada',
74     3: '3: presente y recién aceptada'
75 }
76
77 print('Config OK.')
78 # -----
79
80 # CELDA 3:
81 # -----
82 model = tf.keras.models.load_model(MODEL_PATH)
83 print('Loaded model:', MODEL_PATH)
84 # -----
85
```

```

86 # CELDA 4:
87 # -----
88 # Cargar como audio monofonico a la frecuencia de muestreo (SR)
89 audio, sr = librosa.load(str(WAV_PATH), sr=SR, mono=True)
90
91 print('Loaded WAV:', WAV_PATH.name)
92 print('sr:', sr, 'samples:', len(audio), 'duration (s):', len(audio)/sr)
93 # -----
94
95 # CELDA 5:
96 # -----
97 win_len = int(WINDOW_SEC * SR)
98 hop_len = int(HOP_SEC * SR)
99 smooth_N = int(SMOOTH_SEC / HOP_SEC)
100
101 # Buffer circular para el audio
102 buf = np.zeros(win_len, dtype=np.float32)
103 write_idx = 0
104
105 # Suavizado mediante suma acumulada (running sum)
106 prob_hist = deque(maxlen=smooth_N)
107 prob_sum = np.zeros((4,), dtype=np.float64) # corriendo suma de probabilidades
108
109 times, conf_inst, conf_smooth, pred_inst, pred_smooth = [], [], [], [], []
110
111 t0 = time.time()
112 num_hops = int(np.ceil(len(audio) / hop_len))
113
114 print(f'Running {num_hops} hops | window={WINDOW_SEC}s hop={HOP_SEC}s smooth={SMOOTH_SEC}s ({smooth_N} preds)')
115
116 pos = 0
117 for step in range(num_hops):
118     # Tomar el siguiente fragmento (chunk) del salto (hop)
119     chunk = audio[pos:pos+hop_len]
120     pos += hop_len
121
122     # Rellenar (padding) el ultimo fragmento si es necesario
123     if len(chunk) < hop_len:
124         chunk = np.pad(chunk, (0, hop_len - len(chunk)), mode='constant')
125         chunk = chunk.astype(np.float32, copy=False)
126
127     # escribir en el buffer circular

```

```
128     end = write_idx + hop_len
129     if end <= win_len:
130         buf[write_idx:end] = chunk
131     else:
132         n1 = win_len - write_idx
133         buf[write_idx:] = chunk[:n1]
134         buf[:end - win_len] = chunk[n1:]
135     write_idx = (write_idx + hop_len) % win_len
136
137     # leer los ultimos 2s en el orden correcto (evitar exceso de roll/copy)
138     if write_idx == 0:
139         window = buf
140     else:
141         window = np.concatenate([buf[write_idx:], buf[:write_idx]])
142
143
144     probs = model.predict(x, verbose=0)[0]          # (4,)
145     y_hat = int(np.argmax(probs))
146     c_hat = float(np.max(probs))
147
148     # actualizar el suavizado de la media movil
149     if len(prob_hist) == prob_hist.maxlen:
150         prob_sum -= prob_hist[0]
151     prob_hist.append(probs)
152     prob_sum += probs
153     probs_s = (prob_sum / len(prob_hist)).astype(np.float32)
154
155     y_s = int(np.argmax(probs_s))
156     c_s = float(np.max(probs_s))
157
158     # STATE.update({'t': float(sec), 'class': int(y_s), 'confidence': float(c_s)
159     # })
160
161     sec = step * HOP_SEC
162     times.append(sec)
163     conf_inst.append(c_hat)
164     conf_smooth.append(c_s)
165     pred_inst.append(y_hat)
166     pred_smooth.append(y_s)
167
168     if step % 5 == 0: # imprimir cada 5 segundos para reducir spam/sobrecarga
169         print(f't={sec:6.1f}s - inst={CLASS_NAME[y_hat]:28s} conf={c_hat:0.3f} -
170             smooth={CLASS_NAME[y_s]:28s} conf={c_s:0.3f}')
```



```
169
170     if SIMULATE_REALTIME:
171         elapsed = time.time() - t0
172         target = (step + 1) * HOP_SEC
173         sleep_time = target - elapsed
174         if sleep_time > 0:
175             time.sleep(sleep_time)
176
177 print('Done.')
```

Listing A.4: Extracción de características.